DILATATIONS OF CATEGORIES, VIA THEIR LEAN FORMALIZATION

ARNAUD MAYEUX

ABSTRACT. Given a category $\mathcal{C}$ and a center, that is a collection of pairs (d_i, N_i) consisting of a morphism d_i and a sieve N_i over its codomain, the dilatation of $\mathcal{C}$ is a new category $\mathcal{C}'$ in which every $n \in N_i$ factors, uniquely and functorially, through d_i. This construction refines the classical localization of categories. This paper presents the theory of dilatations of categories through a full formalization of the construction and its main theorems in the Lean 4 proof assistant, on top of the Mathlib library. In particular, we formalize the universal property of dilatations, the operations of restricting, shrinking, and combining centers, the dual theory of codilatations, and the comparison with dilatations of commutative rings. The formalization also produced minor clarifications and corrections to the published theory, recorded in an erratum. An appendix collects a systematic dictionary between the mathematical statements and the Lean declarations that formalize them.

CONTENTS

1. Introduction 2
2. The classical localization of a category, as reused from Mathlib 4
3. Centers 5
4. Building the dilatation as a quotient of a generated category 7
5. The canonical functor and its universal property 10
6. Restricting a center 17
7. Shrinking the sieves 18
8. Combining two centers 19
9. Codilatations 25
10. Comparison with dilatations of rings 26
11. A characterization that fails for categories 29
Appendix A. Dictionary between the mathematics and the Lean formalization 31
Appendix B. Erratum to [5] 42
References 43

Date: August 8, 2026.

Key words and phrases. Dilatations of categories, localization of categories, Lean 4, formalized mathematics, formalized mathematical data, automated deduction.

The Lean 4 source is available at https://github.com/rndmx/DilCat.

1. Introduction

In [6] we considered the benchmark problem of formalizing all indexed mathematics as a machine-verifiable, continuously updated corpus of mathematical knowledge, which would in particular provide large-scale data for AI-assisted mathematics. The present paper is one step in that direction: we formalize a complete published mathematical paper and document the formalization entirely, including the minor clarifications and corrections that the process produced, as anticipated in [6, §5.6]. In this sense, the present paper is also intended as training material for AI systems: informal statements paired with their formal counterparts, machine-checked.

Category theory has become one of Mathlib's largest and most actively developed libraries, and constructions once confined to informal exposition now have precise, checkable counterparts a proof assistant can verify line by line. This paper presents the mathematics of dilatations of categories from their Lean formalization.

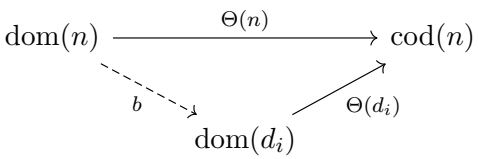
1.1. **Dilatations, informally.**

Let A be a commutative ring and $S \subset A$ a subset. The localization $S^{-1}A$ is obtained by formally adjoining an inverse to every element of S. Localization is a pervasive operation, and its categorical analogue is equally pervasive: given a category $\mathcal{C}$ and a collection Σ of morphisms of $\mathcal{C}$, the localization $\mathcal{C}[\Sigma^{-1}]$ is the category obtained by formally adjoining an inverse to every morphism in Σ, universal among functors out of $\mathcal{C}$ that invert Σ.

Dilatations of rings are a different, more refined operation. Instead of inverting an element $a \in A$ outright, one prescribes a pair (M, a) with M an ideal and a an element of A, and forms a ring in which every element of M becomes divisible by a, but only those elements, and only by a. Dilatations of rings are the local building blocks of dilatations of schemes (Néron blowups and their relatives) [4, 7, 1], and they specialize to localization when M is taken to be the whole ring.

The starting observation of the theory developed here is that both constructions are instances of a single categorical construction. Fix a category $\mathcal{C}$. A center in $\mathcal{C}$ is a family $\{(d_i, N_i)\}_{i \in I}$ where each d_i is a morphism of $\mathcal{C}$ and each N_i is a sieve over the codomain of d_i, i.e. a collection of morphisms into $\mathrm{cod}(d_i)$ closed under precomposition. The dilatation of $\mathcal{C}$ with center $\{(d_i, N_i)\}_{i \in I}$ is a category $\mathcal{C}'$, equipped with a functor $\Theta : \mathcal{C} \to \mathcal{C}'$ that is the identity on objects, such that for every $i \in I$ and every $n \in N_i$ there is a unique morphism b making the triangle

(1)



commute in $\mathcal{C}'$. Informally, b is the "fraction" $d_i \backslash n = d_i^{-1} \circ n$: $\Theta(n)$ has been made to factor through $\Theta(d_i)$. Making this triangle condition into a genuine universal property, so that $\mathcal{C}'$ is determined up to unique isomorphism and functorial in an appropriate sense, is the technical heart of the theory, and occupies Theorem 5.8 below.

1.2. **This paper and its formalization.** Every definition and every theorem in this text is accompanied by its Lean 4 [9] formalization, carried out on top of the Mathlib library [3] and contained in a single file, which we refer to throughout as the formalization. Our governing principle is that the formalization is the primary source: where the original exposition of this material and the Lean development differ (in how a definition is formulated, in the order in which results are proved, in which auxiliary facts are extracted, or occasionally in the precise hypotheses of a theorem), we follow the formalization, and we explain the discrepancy when it is instructive. This is not merely a stylistic choice. Writing a mathematical argument in a form a proof assistant will accept forces every implicit step to be made explicit, and in two places this exposes a gap between what is asserted in the printed paper and what has actually been proved. Both gaps are minor. For [5, Proposition 3.15] (discussed in §8.2 below), one step of the printed proof is not rigorous; the statement is formalized here under an additional hypothesis, and closing the gap unconditionally remains open. For [5, Proposition 5.1] (discussed in §10.2 below), the printed statement is factually wrong as stated — the formalization exhibits an explicit counterexample — but the defect is minor and easily fixed by reindexing the center (§10.1), after which the intended identification holds.

A systematic, tabulated correspondence between every mathematical statement of this paper and the Lean declaration(s) that formalize it is collected in Appendix A, for readers who want to locate a specific statement in the source file quickly; the body of the paper is written to be read on its own; the dictionary is a reference, not a required companion.

1.3. **Organization.** The formalization is organized, roughly, into: a reuse of Mathlib's existing machinery for the classical localization of a category (§2); the definition of a center and of the dilatation it generates, built as a quotient of a freely generated category rather than directly as a set of equivalence classes of "fraction sequences" (§3–§4); the basic properties of the canonical functor $\Theta : \mathcal{C} \to \mathcal{C}'$ (§5.1); the universal property of the dilatation, which is the technical core of the theory (§5); three constructions that produce new dilatations from old ones, by restricting to a sub-family of the center (§6), by shrinking each sieve to a sub-sieve while keeping the same family of morphisms (§7), or by combining two centers into one (§8); the dual notion of codilatation, obtained for free by passing to the opposite category (§9); and finally two worked examples, dilatations of commutative rings (§10) and an explicit finite counterexample showing that a natural strengthening of the theory, which does hold for rings, fails for categories (§11). We follow this

order, which is the order of the formalization, rather than the order of the original paper on this subject, because each construction genuinely depends on the Lean infrastructure built in the sections that precede it.

1.4. **Conventions.** Composition is written in the mathematician's order: $g \circ f$ denotes "f then g". Mathlib, and hence every Lean snippet in this paper, composes in the opposite, diagrammatic order: for `f : X ⟶ Y` and `g : Y ⟶ Z`, the Lean expression `f ≫ g` denotes the composite that we write $g \circ f$. We flag this translation once here and then use it silently throughout: a displayed equation $h = g \circ f$ always corresponds to a Lean equation `h = f ≫ g`.

2. The classical localization of a category, as reused from Mathlib

2.1. **Sieves and the localization construction.** Two pieces of classical category theory are needed: sieves, and the localization of a category at a collection of morphisms. Both are treated, in the formalization, as already available: Mathlib provides them, and the formalization does not reformalize either from scratch. Since a reader of this paper may not be a reader of Mathlib, we describe both constructions here, in the informal language of the original theory, and then explain precisely which pieces of Mathlib play which role.

A sieve N over an object Y of $\mathcal{C}$ is a collection of morphisms with codomain Y, closed under precomposition by an arbitrary morphism of $\mathcal{C}$. Given a collection E of morphisms of $\mathcal{C}$, the sieve S_E generated by E consists of all composites $e \circ n$ with $e \in E$ and n an arbitrary morphism composable with e; when $E = \{d\}$ is a single morphism we write S_d for $S_{\{d\}}$. In Lean, `Sieve X` is a structure recording, for every object Y, a set of morphisms $Y \to X$ satisfying the closure condition, and `Sieve.generate` builds S_E from a `Presieve` (an arbitrary, not necessarily closed, family of morphisms) representing E.

Given a collection Σ of morphisms of $\mathcal{C}$, the localization $\mathcal{C}[\Sigma^{-1}]$ is built classically (Gabriel–Zisman [2]) as follows: form the directed graph $\mathcal{G}$ with one vertex per object of $\mathcal{C}$, one edge a for every morphism a of $\mathcal{C}$, and one formal inverse edge $\ell_d : \mathrm{cod}(d) \to \mathrm{dom}(d)$ for every $d \in \Sigma$; a Σ-sequence is a finite path in $\mathcal{G}$; two Σ-sequences are declared equivalent if one can be obtained from the other by composing two consecutive edges of $\mathcal{C}$, by cancelling a consecutive d, ℓ_d or ℓ_d, d pair, or by deleting an identity edge; and $\mathcal{C}[\Sigma^{-1}]$ is the category whose morphisms are equivalence classes of Σ-sequences. This is a completely general construction, needing nothing about $\mathcal{C}$ beyond its being a category, and it satisfies the expected universal property: a functor $F : \mathcal{C} \to \mathcal{D}$ that inverts every $d \in \Sigma$ factors uniquely through the canonical functor $L : \mathcal{C} \to \mathcal{C}[\Sigma^{-1}]$.

This is, verbatim, the construction implemented in Mathlib's `CategoryTheory.Localization.Construction` file: the graph $\mathcal{G}$ is

`Localization.Construction.LocQuiver`, Σ-sequences are paths in this quiver (Mathlib's general-purpose `Quiver.Path` type, via the free/path category `CategoryTheory.Paths`), the equivalence relation of [5, Definition 2.4] is `Localization.Construction.relations` (a congruence on paths in the sense of §4 below), and the localization itself is presented as `(W).Localization` for `W` a `MorphismProperty` (Mathlib's name for a collection of morphisms closed under no particular condition, simply a predicate `∀ {{X Y}}, (X ⟶ Y) → Prop`), together with a canonical functor `W.Q` playing the role of L, and the universal property is `Localization.Construction.lift/fac/uniq`.

Design choice. Rather than reproduce Mathlib's localization construction adapted to Σ-sequences, the formalization builds the raw localization $\mathcal{C}[\Sigma^{-1}]$ once and defines the dilatation as a further quotient mapping to it (§4). This gives associativity of Σ-sequence composition, and every general Mathlib fact about localizations, for free.

3. Centers

3.1. The data of a center.

Definition 3.1 (Center)**.** A center in a category $\mathcal{C}$ is a family $\{(d_i, N_i)\}_{i\in I}$, indexed by a nonempty type I, such that for every $i \in I$, d_i is a morphism of $\mathcal{C}$ and N_i is a sieve over $\mathrm{cod}(d_i)$.

A center bundles exactly the data needed to state the factorization condition of (1): an index set, and for each index a morphism together with a sieve over its codomain. In Lean this is a single structure:

```
structure Center (C : Type u) [Category.v C] where
I : Type u
(nonempty :  Nonempty I)
dom :  I → C
cod :  I → C
mor :  ∀ i :  I, dom i ⟶ cod i
N : ∀ i :  I, Sieve (C := C) (cod i)
```

Design choice. Two encoding choices are worth pointing out. First, `dom` and `cod` are recorded as separate functions $I \to \mathcal{C}$ rather than being read off from `mor i`: this is purely for convenience of use (so that, e.g., `Z.dom i` can appear in a statement without first destructuring `Z.mor i`), and is definitionally redundant with `mor`, whose type `dom i ⟶ cod i` already determines both. Second, the family is recorded as a structure with a function-valued field `I → ...` (a Mathlib-style indexed family) rather than, say, a `Set` of triples (X, Y, f) with a sieve attached to each: this is again the standard idiom for "an I-indexed collection of data" in Lean, and it is what makes the disjoint union of two centers (§8) a one-line definition, gluing two index types with `Sum`.

We write Z for a center, and, following the paper this material is drawn from, write $\Sigma = \{d_i\}_{i \in I}$ for the underlying collection of morphisms (forgetting the sieves). Membership in Σ is not recorded as a `Set` in Lean; instead, being a central morphism is a property of an arbitrary triple (X, Y, f):

```
def IsCenterMor (f : Σ X Y : C, X ⟶ Y) : Prop :=
∃ i : Z.I, f = ⟨Z.dom i, Z.cod i, Z.mor i⟩

def CenterMorphismProperty : MorphismProperty C :=
fun X Y f => IsCenterMor Z ⟨X, Y, f⟩
```

Here `Σ X Y : C, X ⟶ Y` is Lean's dependent-pair (sigma) type: an element bundles an object X, an object Y, and a morphism $X \to Y$ into one piece of data, giving a uniform way to talk about "an arbitrary morphism of $\mathcal{C}$," regardless of its endpoints, as a single mathematical object one can quantify over or compare for equality. This idiom recurs (Definition 4.2). `MorphismProperty C` is Mathlib's type of predicates on morphisms of $\mathcal{C}$ (formally, `∀ {{X Y}}, (X ⟶ Y) → Prop`), exactly the data needed to describe "a collection of morphisms" without committing to how that collection is presented; `CenterMorphismProperty` is the `MorphismProperty` cut out by Σ. Feeding this `MorphismProperty` to Mathlib's localization construction from §2 gives, with no further work, the raw localization $\mathcal{C}[\Sigma^{-1}]$ and its canonical functor:

```
def CenterLocalization : Type u := (CenterMorphismProperty
Z).Localization

def LocalizationFunctor : C ⥤ (CenterMorphismProperty
Z).Localization :=
(CenterMorphismProperty Z).Q
```

This localization $\mathcal{C}[\Sigma^{-1}]$, inverting every d_i and forgetting every N_i, is not the dilatation; it will however reappear constantly, in two roles: as the target of a faithfulness hypothesis controlling uniqueness in the universal property (§5), and, in the special case where every N_i is taken to be the largest possible sieve, as the dilatation itself:

Fact 3.2 ([5, Fact 2.15]). *If $N_i = S_{\mathrm{id}_{\mathrm{cod}(d_i)}}$ (the sieve of every morphism into $\mathrm{cod}(d_i)$) for every $i \in I$, then the dilatation $\mathcal{C}[\{(d_i)^{-1} \circ N_i\}_{i \in I}]$ is canonically isomorphic to the raw localization $\mathcal{C}[\Sigma^{-1}]$.*

Taking every N_i maximal forces every morphism into $\mathrm{cod}(d_i)$ to factor through d_i, which forces d_i itself to become invertible: dilating by the maximal sieve is exactly localizing. In Lean, `Center.ofMorphisms` builds the center with this maximal choice of sieve (`N := fun _ => ⊤`), and `Fact_2_15` is the resulting isomorphism of categories; §A.5 of the appendix records how, combined with Theorem 5.8, this recovers [5, Proposition 2.8]'s classical universal property of localization (§2) as a special case of Theorem 5.8 rather than as a separately proved fact.

4. Building the dilatation as a quotient of a generated category

We now fix a center $Z = \{(d_i, N_i)\}_{i\in I}$ in $\mathcal{C}$ and describe how the formalization builds the dilatation $\mathcal{C}'$. This section is the most consequential design decision in the whole formalization: the route taken here is not the route of the original paper, and the difference propagates through every later proof.

4.1. **The cost of formalizing fraction sequences directly.** Informally (and in the original exposition of this material), a morphism of the dilatation $\mathcal{C}'$ is an equivalence class of a $\{(d_i, N_i)\}_{i\in I}$-fraction: a Σ-sequence in the graph $\mathcal{G}$ of §2 of the special shape

$$X_1 \xrightarrow{n_1} Y_1 \xrightarrow{\ell_{d_{i_1}}} X_2 \xrightarrow{n_2} Y_2 \xrightarrow{\ell_{d_{i_2}}} X_3 \cdots X_k \xrightarrow{n_k} Y_k \xrightarrow{\ell_{d_{i_k}}} X_{k+1} \xrightarrow{a} X_{k+2},$$

with a an arbitrary morphism of $\mathcal{C}$, $k \geq 0$, and $n_j \in N_{i_j}$ for every j. Composition of such fractions, and the well-definedness of composition on equivalence classes, are then checked directly, generalizing [5, Fact 2.6]'s computation ($\ell_{d'} \circ \ell_d = \ell_{d''}$ when d, d' compose to a further central morphism d''). Carried out by hand, in Lean, on explicit equivalence classes of sequences, this would require reproving associativity of composition ([5, Fact 2.11] in the original numbering) essentially from scratch, in a shape special enough that none of Mathlib's existing localization machinery applies directly. For a compiling formalization of this approach, see [6] (this paper was not primarily concerned with formalizing dilatations of categories; dilatations of categories were an example of an ongoing formalization effort, and it was indicated there that the formalization choices were not final).

4.2. **The route taken: a freely generated category, then a quotient.** The formalization instead builds $\mathcal{C}'$ in two stages, each reusing a piece of Mathlib's generic category-theoretic infrastructure.

Stage 1: a quiver of generators. Define a quiver (a directed graph, with no composition structure yet) on the objects of $\mathcal{C}$, with two kinds of edges: one original edge $X \to Y$ for every morphism $a : X \to Y$ of $\mathcal{C}$, and one fraction edge $\mathrm{dom}(n) \to \mathrm{dom}(d_i)$ for every $i \in I$ and every $n \in N_i$, namely the data of a pair (i, n), which we call, following §1.3's terminology, a $\{(d_i, N_i)\}_{i\in I}$-fraction generator. This is exactly the data needed to write down one instance of the factorization triangle (1): the fraction edge for (i, n) is a name for the morphism b that triangle asks for.

```
def CenterSievePair :  Type (max u v) :=
Σ i :  Z.I, Σ X : C,  f :  X ⟶ Z.cod i // Z.N i f

inductive GeneratorMorphismData (Z : Center C) X Y :
(CenterMorphismProperty Z).Localization
(f :  X ⟶ Y) : Type (max u v)
| fraction :  PairMorWitness Z f → GeneratorMorphismData Z f
| original :  OriginalWitness Z f → GeneratorMorphismData Z f
```

```
def GeneratorQuiver :  Quiver (CenterMorphismProperty
Z).Localization where
Hom X Y := Σ f :  X ⟶ Y, GeneratorMorphismData Z f
```

A `CenterSievePair` bundles an index i, an object X, and a witness that some $n : X \to \mathrm{cod}(d_i)$ lies in N_i, precisely the data (i, n) indexing a fraction edge. The quiver itself is built, slightly indirectly, on the objects of the raw localization $\mathcal{C}[\Sigma^{-1}]$ rather than directly on the objects of $\mathcal{C}$ (the two are in canonical bijection, via Mathlib's `objEquiv`, since localization is the identity on objects); this lets both kinds of generator edges (an original morphism of $\mathcal{C}$, or a fraction witnessed by a `CenterSievePair`) be compared for equality as morphisms of the raw localization, which is exactly what is needed a few lines below to formulate the congruence of Stage 2. `GeneratorMorphismData Z f` records why a given morphism f of the raw localization arises as a generator: either it is (the image of) an original morphism of $\mathcal{C}$ (constructor `original`), or it is (the image of) a fraction-witness composite $n \circ \ell_{d_i}$ built from a `CenterSievePair` (constructor `fraction`).

That underlying composite is not left informal: it is constructed in three named steps, descending through the levels of Mathlib's localization construction (§2) — first as a path in the path category of the localization quiver, then as its class in the raw localization.

```
 def inv_in_path (p :  CenterSievePair Z) :
ιPaths (CenterMorphismProperty Z) (Z.cod p.1) ⟶ ιPaths
(CenterMorphismProperty Z) (Z.dom p.1) :=
Localization.Construction.ψ₂ (CenterMorphismProperty Z) (Z.mor p.1)
⟨p.1, rfl⟩
```

```
def fraction_in_path_single (p :  CenterSievePair Z) :
ιPaths (CenterMorphismProperty Z) (p.2.1) ⟶ ιPaths
(CenterMorphismProperty Z) (Z.dom p.1) :=
Localization.Construction.ψ₁ (CenterMorphismProperty Z) p.2.2.1 ≫
inv_in_path Z p
```

```
def fraction_in_loc_single (p :  CenterSievePair Z) :
objEquiv (CenterMorphismProperty Z) (p.2.1) ⟶ objEquiv
(CenterMorphismProperty Z) (Z.dom p.1) :=
(CategoryTheory.Quotient.functor (relations (CenterMorphismProperty
Z))).map (fraction_in_path_single Z p)
```

Here ψ_1 and ψ_2 are Mathlib's names (in `Localization.Construction`) for the length-one path on an original edge, respectively on a formal-inverse edge, of the localization quiver `LocQuiver` of [5, Definition 2.2]. So `inv_in_path Z p` is (the length-one path on) the edge ℓ_{d_i} for the inverted morphism d_i, and `fraction_in_path_single Z p` is the path-level composite $n \circ \ell_{d_i}$ (in diagrammatic order, $\psi_1(n)$ followed by `inv_in_path`). Applying the quotient functor of the raw localization yields `fraction_in_loc_single`

`Z p`, the class of this composite in $\mathcal{C}[\Sigma^{-1}]$; this is exactly the morphism that `IsPairMor` and `PairMorWitness` — and through them the constructor `fraction` above — compare against, and it resurfaces in §5.1 as the underlying morphism of the fraction generator edge.

Stage 2: freely generate, then quotient. Mathlib's `CategoryTheory.Paths` functor turns any quiver into a category: the path category, whose morphisms are finite paths of edges, with composition given by concatenation, and associativity holding by construction (concatenation of lists is associative). Applying it to `GeneratorQuiver Z` gives a category `GeneratedCategory Z` whose morphisms are exactly the $\{(d_i, N_i)\}$-fractions of the informal description above, without yet imposing any relations between different paths representing "the same" fraction.

```
def GeneratedCategory := CategoryTheory.Paths (GeneratorObjects Z)
instance : Category (GeneratedCategory Z) := Paths.categoryPaths _
```

To obtain $\mathcal{C}'$, this free category must still be quotiented by the relations that make two representative sequences of the same fraction equal, exactly the informal content of [5, Definition 2.4]'s elementary equivalences, transported to this setting. Rather than re-deriving these relations combinatorially, the formalization defines the congruence directly by reference to the raw localization, whose associated equivalence relation on sequences is already correctly set up by Mathlib (§2):

```
def GeneratedToLocalization : GeneratedCategory Z ⇒
(CenterMorphismProperty Z).Localization :=
CategoryTheory.Paths.lift (forgetGenerator Z)

def DilaRel : HomRel (GeneratedCategory Z) :=
fun _ _ f g => (GeneratedToLocalization Z).map f =
(GeneratedToLocalization Z).map g

def Dila := CategoryTheory.Quotient (DilaRel Z)
instance : Category (Dila Z) := CategoryTheory.Quotient.category _
```

Here `forgetGenerator` is the evident prefunctor forgetting the `GeneratorMorphismData` tag and remembering only the underlying morphism of the raw localization, and `Paths.lift` turns this prefunctor into an honest functor `GeneratedToLocalization Z`, a functor `GeneratedCategory Z` $\to \mathcal{C}[\Sigma^{-1}]$ out of the free category (again by the universal property of the path-category construction: a functor out of a free category on a quiver is the same thing as a prefunctor on that quiver). Two paths are declared `DilaRel`-related exactly when they become equal after mapping to the raw localization, i.e., exactly when they represent the same Σ-fraction, ignoring the extra bookkeeping of which N_i's were used. Finally, `Dila Z` is defined as the quotient category `CategoryTheory.Quotient (DilaRel Z)`, another piece of generic Mathlib infrastructure: given any category and any relation on its `Hom`-sets, `Quotient` produces the quotient category, with composition

well-defined automatically because the construction proves once and for all that the smallest congruence containing a given relation is compatible with composition (the `Congruence` typeclass, instantiated here in three lines by reflexivity/symmetry/transitivity of equality and compatibility of `Functor.map` with composition).

Design choice. This two-stage route (free category on a quiver of generators, then quotient by "agreement after forgetting to the raw localization") trades a bespoke combinatorial argument (associativity and well-definedness of fraction composition, checked by hand on representative sequences) for two invocations of general-purpose machinery (`Paths.categoryPaths` for associativity of the free category, `Quotient.category` for well-definedness of the quotient), at the cost of introducing an auxiliary category, `GeneratedCategory Z`, that has no name in the original exposition. This auxiliary category is not merely a bookkeeping device: since `Dila Z` is a quotient of it and quotient functors are full onto their generators, Theorem 5.8 is proved by induction on it, via `GeneratedCategory_morphism_induction`.

4.3. **The comparison with the raw localization is faithful.** Writing $\mathcal{C}'$ for `Dila Z`, the functor `GeneratedToLocalization Z` used to define `DilaRel` descends, by construction, to a functor `DilaToLoc Z`, of type $\mathcal{C}' \to \mathcal{C}[\Sigma^{-1}]$ (forgetting, once again, the sieve data and remembering only which morphisms of $\mathcal{C}$ were inverted).

Fact 4.1 ([5, Fact 2.14])**.** *The functor* `DilaToLoc Z` $: \mathcal{C}' \to \mathcal{C}[\Sigma^{-1}]$ *is faithful.*

This is the first structural fact about $\mathcal{C}'$, and it already illustrates the payoff of the two-stage construction: since two morphisms of $\mathcal{C}'$ agree if and only if they are represented by paths that are `DilaRel`-related, i.e. paths with the same image in $\mathcal{C}[\Sigma^{-1}]$ by definition of `DilaRel`, faithfulness of `DilaToLoc Z` is close to a tautology once phrased this way: given f, g with the same image, `Quot.sound` (Lean's principle that provably-related representatives of a quotient give equal classes) directly produces $f = g$ in $\mathcal{C}'$. The formalized proof is a few lines unfolding the quotient's `Hom`-type as a `Quot` of paths and invoking exactly this principle. Contrast this with proving faithfulness directly from an explicit description of fraction composition, which would require re-deriving that $\mathcal{C}'$'s equivalence classes inject into $\mathcal{C}[\Sigma^{-1}]$'s, again the associativity argument that §4.2 exchanges for machinery.

5. The canonical functor and its universal property

5.1. **The canonical functor and the fraction morphisms.** Every object of $\mathcal{C}$ is, tautologically, an object of $\mathcal{C}' =$`Dila Z` (both categories share the same objects as $\mathcal{C}[\Sigma^{-1}]$ and as the raw generated category); every morphism a of $\mathcal{C}$ gives, via the "original" generator edge it defines, a morphism of $\mathcal{C}'$. This assembles into a functor.

Proposition 5.1 ([5, Proposition 3.1(i)])**.** *There is a canonical functor $\Theta : \mathcal{C} \to \mathcal{C}'$, the identity on objects, sending a morphism a of $\mathcal{C}$ to the class of the length-one path on the corresponding original generator edge.*

In Lean this is `CatToDila Z`, built directly from the quiver embedding `CToGeneratorQuiver` of §4, composed with the quotient functor `Quotient.functor (DilaRel Z)`; functoriality (respecting identities and composition) is checked by unfolding both sides in the raw localization $\mathcal{C}[\Sigma^{-1}]$, where it is immediate.

The second half of [5, Proposition 3.1] is the existence, for every $i \in I$ and $n \in N_i$, of the fraction morphism b solving the triangle (1), and this is, by construction, exactly what a fraction generator edge is.

Proposition 5.2 ([5, Proposition 3.1(ii)])**.** *For every $i \in I$, $n \in N_i$, there is a (unique, see Theorem 5.8 below) morphism $b : \mathrm{dom}(n) \to \mathrm{dom}(d_i)$ in $\mathcal{C}'$ with $\Theta(d_i) \circ b = \Theta(n)$.*

```
def fraction_in_dila_single (p :  CenterSievePair Z) :
(CatToDila Z).obj p.2.1 ⟶ (CatToDila Z).obj (Z.dom p.1) :=
(GeneratedToDila Z).map (Quiver.Hom.toPath (fraction_in_generated Z
p))
```

i.e. b is, once again, literally a name for the corresponding generator edge, viewed in $\mathcal{C}'$ instead of in the free category. Unwinding the definitions, this is the top of the ladder begun in §4.2: `fraction_in_generated Z p` is the generator edge whose underlying raw-localization morphism is `fraction_in_loc_single Z p`, itself the class of the path-level composite `fraction_in_path_single Z p`, i.e. of ψ_1 `n` $\gg$ `inv_in_path Z p` — so the chain `inv_in_path` → `fraction_in_path_single` → `fraction_in_loc_single` → `fraction_in_generated` → `fraction_in_dila_single` consists of successive reinterpretations of the single composite $n \circ \ell_{d_i}$, ending in $\mathcal{C}'$. The defining equation $\Theta(d_i) \circ b = \Theta(n)$ (`fraction_in_dila_comp_mor`) is proved by `Quotient.sound`, reducing to a one-line computation in the raw localization exactly as in Fact 4.1. Uniqueness of b is deliberately not proved at this point in the formalization: it is deferred to, and falls out of, Theorem 5.8, since it is a special case of the uniqueness half of the universal property applied to $F = \Theta$ itself. This is a first small instance of a recurring pattern: rather than proving uniqueness statements piecemeal, the formalization proves the strongest available uniqueness principle once (Theorem 5.8) and specializes it repeatedly.

Recall that a bimorphism is a morphism that is both mono and epi.

Fact 5.3 ([5, Fact 3.2])**.** *The image of a morphism under a faithful functor being a bimorphism forces the morphism itself to be a bimorphism.*

This is a general categorical fact, with a two-line proof by cancellation (if $F(f)$ is mono and $a \circ f = b \circ f$ then $F(a) \circ F(f) = F(b) \circ F(f)$, so $F(a) = F(b)$ by cancelling the mono $F(f)$, so $a = b$ by faithfulness; dually

for epi), formalized as `Fact_3_2` for an arbitrary faithful F, not specialized to Θ.

Proposition 5.4 ([5, Proposition 3.3]). *For every $i \in I$, $\Theta(d_i)$ is a bimorphism in $\mathcal{C}'$.*

Indeed `DilaToLoc Z` is faithful (Fact 4.1) and sends $\Theta(d_i)$ to an isomorphism of $\mathcal{C}[\Sigma^{-1}]$ (localizations invert every generator by construction), and isomorphisms are bimorphisms; [5, Fact 3.2] concludes.

5.2. **Sieve inclusion and Σ-regularity.** Two more ingredients are needed before the universal property can be stated: a sieve inequality tracking how Θ interacts with the sieves N_i, and a faithfulness condition on the target functor F playing the role that faithfulness of `DilaToLoc Z` plays for Θ itself.

For $i \in I$, write $S_{\Theta(N_i)}$ for the sieve over $\Theta(\mathrm{cod}(d_i))$ generated by $\{\Theta(n) \mid n \in N_i\}$, and $S_{\Theta(d_i)}$ for the sieve generated by the single morphism $\Theta(d_i)$.

Proposition 5.5 ([5, Proposition 3.5]). *For every $i \in I$, $S_{\Theta(N_i)} \subset S_{\Theta(d_i)}$.*

This says precisely that every morphism of $S_{\Theta(N_i)}$ (a composite $\Theta(n) \circ t$ with $n \in N_i$) factors through $\Theta(d_i)$; and indeed it does, through $b \circ t$ with b the fraction morphism of Proposition 5.2. In Lean, `CatToDila_image_sieve_le_singleton` proves this by producing this explicit factorization: unfolding membership in a pushforward sieve (Mathlib's `Sieve.functorPushforward`, used here to express $S_{\Theta(N_i)}$ as the pushforward of N_i along Θ) down to a witness $(Y, h, g, hg, \mathrm{rfl})$, and exhibiting $g \circ$`fraction_in_dila_single`(i, h) as the required factorization, via the `calc` chain

$$(g \circ b) \circ \Theta(d_i) = g \circ (b \circ \Theta(d_i)) = g \circ \Theta(h).$$

Definition 5.6 ([5, Definition 3.6]). A functor $F : \mathcal{C} \to \mathcal{D}$ is Σ-regular if the canonical functor $\mathcal{D} \to \mathcal{D}[F(\Sigma)^{-1}]$ is faithful.

In Lean, Σ-regularity is recorded as a `Prop`, `IsSigmaRegular Z F`, defined as faithfulness of `(ImageCenterMorphismProperty Z F).Q`, where `ImageCenterMorphismProperty Z F` is the collection of morphisms of $\mathcal{D}$ that are images under F of some d_i. This is deliberately not framed as membership in a comma category $Cat_{\mathcal{C}}^{\Sigma\text{-reg}}$ (the original exposition's route): the formalization never needs to talk about morphisms between Σ-regular functors, only about individual functors being Σ-regular, so recording only the property, not the category structure around it, avoids building infrastructure that is never used.

Fact 5.7 ([5, Fact 3.7]). *$\Theta : \mathcal{C} \to \mathcal{C}'$ is Σ-regular.*

Since Θ inverts, in $\mathcal{C}'$, exactly the same morphisms of $\mathcal{C}'$ that `DilaToLoc Z` inverts (both localizations at the images of $\{d_i\}$ under a functor identifying $\mathcal{C}'$ and, further down the line, $\mathcal{C}[\Sigma^{-1}]$, agree), this follows formally from Fact 4.1

together with the following purely categorical lemma, which is stated once and used twice:

```
theorem faithful_of_comp_faithful
(p : C₁ ⇒ C₂) (e : C₂ ⇒ C₃) (hfaith : (p ⋙ e).Faithful) :
p.Faithful := by
constructor
intro X Y f g h
apply hfaith.map_injective
simp only [Functor.comp_map, h]
```

Design choice. This lemma ("a functor that factors, on the target side, through a faithful functor is itself faithful") is the single fact driving both [5, Fact 3.7] above and the following fact, which does not otherwise appear as a named result in this section but is used repeatedly in §8:

> If $\Sigma' \subset \Sigma$ and the localization $\mathcal{C} \to \mathcal{C}[\Sigma^{-1}]$ is faithful, then $\mathcal{C} \to \mathcal{C}[\Sigma'^{-1}]$ is also faithful.

The original exposition proves this by a triangle-of-functors argument specific to localizations ($\mathcal{C}[\Sigma^{-1}] \cong \mathcal{C}[\Sigma'^{-1}][\dots]$, then cancel). The formalization instead isolates the one-line categorical fact both arguments really rest on, and proves it once, in a universe-polymorphic form (`faithful_of_comp_faithful_gen`) so that it applies uniformly regardless of which universes the three categories involved happen to live in.

5.3. **The universal property.** We can now state the theorem that makes $\mathcal{C}'$ the dilatation of $\mathcal{C}$ with center Z, rather than merely a category equipped with a functor satisfying the factorization triangle (1).

Theorem 5.8 ([5, Theorem 3.10])**.** *Let $F : \mathcal{C} \to \mathcal{D}$ be a functor such that*

(1) for every $i \in I$, $S_{F(N_i)} \subset S_{F(d_i)}$ (the sieve generated by $\{F(n) \mid n \in N_i\}$ is contained in the sieve generated by $F(d_i)$ alone), and
(2) F is Σ-regular (Definition 5.6).

Then there is a unique functor $F' : \mathcal{C}' \to \mathcal{D}$ with $F' \circ \Theta = F$.

Condition (1) is exactly what is needed for existence: it says every generator edge of $\mathcal{C}'$ can plausibly be sent somewhere in $\mathcal{D}$ compatibly with F. Condition (2) is exactly what is needed for uniqueness: it says $\mathcal{D}$ cannot "accidentally" identify two things that a correct choice of F' would have to keep apart. In Lean:

```
theorem Dila_universal_property
(F : C ⇒ D)
(hfaith : (ImageCenterLocalizationFunctor Z F).Faithful)
(hsieve : ∀ (i : Z.I),
Sieve.functorPushforward F (Z.N i) ≤
Sieve.generate (Presieve.singleton (F.map (Z.mor i)))) :
∃! (G : Dila Z ⇒ D), CatToDila Z ⋙ G = F
```

with `hfaith` phrased, as in Definition 5.6, via the auxiliary localization `ImageCenterLocalizationFunctor Z F` (the localization of $\mathcal{D}$ at the images of the d_i's under F), and `hsieve` the Lean transcription of condition (1), using `Sieve.functorPushforward` for "the sieve generated by the image of N_i."

Design choice. The theorem is stated with (1) and (2) as two separate, named hypotheses, rather than as "F is an object of $Cat_{\mathcal{C}}^{\Sigma\text{-reg}}$ satisfying the sieve inequalities," continuing the choice already made in Definition 5.6 not to formalize the comma category itself. Nothing is lost: an F satisfying both hypotheses is exactly an object of $Cat_{\mathcal{C}}^{\Sigma\text{-reg}}$ satisfying the sieve inequalities, and every use of the theorem below supplies the two hypotheses directly rather than by first constructing a comma-category object and then checking the sieve inequalities for it.

The proof splits, as it must, into existence and uniqueness, and (this is the payoff of §4's two-stage construction) both halves are organized around the same induction principle, which we state first.

Lemma 5.9 (Induction on the generated category)**.** *Let P be a property of morphisms of `GeneratedCategory Z`. If P holds of every identity, is closed under composition, and holds of every single generator edge, then P holds of every morphism.*

```
lemma GeneratedCategory_morphism_induction
(P : ∀ X Y : GeneratedCategory Z, (f :  X ⟶ Y) → Prop)
(h_id :  ∀ X, P (𝟙 X))
(h_comp :  ∀ X Y W (f :  X ⟶ Y) (g :  Y ⟶ W), P f → P g → P (f
≫ g))
(h_gen :  ∀ A B : GeneratorObjects Z (g :  (GeneratorQuiver Z).Hom A
B),
P (Quiver.Hom.toPath g)) :
∀ X Y : GeneratedCategory Z (f :  X ⟶ Y), P f
```

This is Mathlib's own induction principle for the free category on a quiver (`CategoryTheory.Paths.induction`: every path is empty, or a shorter path plus one edge), restated via $\ggg$ instead of Mathlib's internal `cons` representation. Since `GeneratorQuiver Z`'s edges are exactly the two kinds of generators of §4, an equivalent restatement of Lemma 5.9 is: to check a property closed under composition and holding on identities, it suffices to check it on every $\Theta(a)$, $a \in \operatorname{Mor}\mathcal{C}$, and on every fraction morphism b. This restatement is what is actually invoked at each of the three call sites below.

5.3.1. *Existence.* The construction of F' proceeds generator by generator. On an original generator, there is only one sensible choice: send $\Theta(a) \mapsto F(a)$. On a fraction generator (i, n), condition (1) says $F(n)$ lies in the sieve $S_{F(d_i)}$, i.e. $F(n) = q \circ F(d_i)$ for some q; condition (2) (via [5, Fact 3.2],

applied to the faithful localization functor of condition (2)) makes $F(d_i)$ a bimorphism, hence q is unique. This local, generator-by-generator choice is fixed first:

```
def uniqueFactor_D (hfaith : ...) (hsieve : ...) (i : Z.I) (Y
: C)
(n : Y ⟶ Z.cod i) (hn : Z.N i n) : F.obj Y ⟶ F.obj (Z.dom i)
:=
Classical.choose (exists_unique_factor_D Z F hfaith hsieve i (F.obj
Y) (F.map n) ...)
```

and then assembled into a prefunctor `Gq` on `GeneratorQuiver Z` (sending an original edge to $F(a)$ and a fraction edge (i, n) to `uniqueFactor_D` applied to (i, n)), which Mathlib's `Paths.lift` turns into an honest functor H :`GeneratedCategory Z`$\to \mathcal{D}$ out of the free category, the same universal property of free categories used to build `GeneratedToLocalization` in §4, now used to build H instead.

It remains to descend H along the quotient by `DilaRel`, i.e., to check that H sends `DilaRel`-related paths to equal morphisms of $\mathcal{D}$, and this is where condition (2) is used a second time, more globally. The key intermediate step is:

> H followed by the localization $\mathcal{D} \to \mathcal{D}[F(\Sigma)^{-1}]$ agrees, as a functor, with `GeneratedToLocalization Z` followed by the comparison functor $\mathcal{C}[\Sigma^{-1}] \to \mathcal{D}[F(\Sigma)^{-1}]$ induced by F.

This is `generatedLocalization_commutes`, proved by Lemma 5.9: on identities and composites it is immediate; on an original generator it reduces to the functoriality of the comparison functor; and on a fraction generator (i, n) it reduces, after unwinding both sides, to a cancellation against the (now invertible, in the further localization) image of $F(d_i)$, the same bimorphism-cancellation idea as Proposition 5.5, one level further down. Given this compatibility, two `DilaRel`-related paths f, g (i.e. with the same image in $\mathcal{C}[\Sigma^{-1}]$) have the same image under H followed by localization; since that localization is faithful (condition (2)), $H(f) = H(g)$ in $\mathcal{D}$ outright. This descent condition is recorded as a standalone lemma, `H_descends`, and Mathlib's `CategoryTheory.Quotient.lift` then descends H to a named functor — the formalization's incarnation of F' itself:

```
def DilaLift (hfaith : ...) (hsieve : ...) : Dila Z ⥤ D :=
CategoryTheory.Quotient.lift (DilaRel Z) (H Z F hfaith hsieve)
(fun _ _ f g hfg => H_descends Z F hfaith hsieve f g hfg)
```

with `GeneratedToDila`$\ggg$`DilaLift`$= H$ by construction; unwinding definitions on generators recovers the defining equation $F' \circ \Theta = F$, recorded as `DilaLift_fac`. Because F' is thus a structural definition (a quotient-lift of a path-lift) rather than a witness extracted from an existential, it is directly reusable downstream: the comparison functors `restrictPhi`, `Phi315`, and `Alpha'315` of §6 and §8 are literally `DilaLift` applied to the relevant

F, with their defining equations given by `DilaLift_fac` rather than re-extracted from an existential at each use.

Design choice. Condition (2) is needed already in the proof of existence, not only of uniqueness: producing $G : \mathcal{C}' \to \mathcal{D}$ requires knowing H respects the quotient relation "agrees after further localizing," itself a faithfulness statement about the localization on the $\mathcal{D}$ side.

5.3.2. *Uniqueness.* Suppose $G_1, G_2 : \mathcal{C}' \to \mathcal{D}$ both satisfy $\Theta \ggg G_i = F$. They agree on objects (both restrict to F on objects of $\mathcal{C}$, and every object of $\mathcal{C}'$ is $\Theta(X)$ for some X). To show they agree on morphisms, Lemma 5.9 is invoked once more, transported along $\mathcal{C}'$: since the quotient functor `GeneratedToDila` is full (a general fact about Mathlib's `Quotient` construction), every morphism of $\mathcal{C}'$ is the image of some morphism of `GeneratedCategory Z`, so it suffices to check G_1, G_2 agree (up to the evident transport along the object-level equality) on images of the two kinds of generators:

- on $\Theta(a)$ for $a \in \operatorname{Mor}\mathcal{C}$: immediate, both sides equal $F(a)$;
- on a fraction morphism b (for (i, n)): here the argument is exactly the one sketched in §5.1 for Proposition 5.2's uniqueness. From $b \circ \Theta(d_i) = \Theta(n)$, applying G_j gives $G_j(b) \circ G_j(\Theta(d_i)) = G_j(\Theta(n)) = F(n)$; since $G_j(\Theta(d_i)) = F(d_i)$ by hypothesis, and $F(d_i)$ is mono (condition (2), via [5, Fact 3.2] again), the two equations $G_1(b) \circ F(d_i) = F(n) = G_2(b) \circ F(d_i)$ cancel to $G_1(b) = G_2(b)$.

This two-case check is recorded as `Generated_factor_unique_map`, and Lean's `Functor.ext` (two functors are equal if they agree on objects and, up to transport, on morphisms) finishes the proof as `Dila_factor_unique`. Specialized to `DilaLift`, this gives `DilaLift_unique` (any G with $\Theta \ggg G = F$ equals `DilaLift`), and the Lean statement of Theorem 5.8 displayed above, `Dila_universal_property`, is then nothing more than the pair (`DilaLift_fac`, `DilaLift_unique`) packaged as an $\exists!$.

Design choice. Uniqueness here and uniqueness of the fraction morphism b (Proposition 5.2) are proved by the same cancellation-against-a-mono argument, applied to $F(d_i)$ and to $\Theta(d_i)$ respectively; the latter is exactly the $F = \Theta$ instance of the former.

5.4. **A representability corollary.** Restating Theorem 5.8 as a bijection statement is immediate and convenient for the applications of §6–§8.

Proposition 5.10 ([5, Proposition 3.12])**.** *For F a Σ-regular functor, there exists a (necessarily unique) F' with $F' \circ \Theta = F$ if and only if $S_{F(N_i)} \subset S_{F(d_i)}$ for every $i \in I$.*

The forward direction is Proposition 5.5 pushed forward along F' ([5, Fact 3.11]: pushforward of a generated sieve along a further functor composes correctly, a routine but essential lemma about `Sieve.functorPushforward` used here and, again, in §8); the backward direction is Theorem 5.8 itself.

In Lean, `CatToDila_represents` states exactly this "iff," under the standing assumption that F is Σ-regular, which is more directly usable than the original phrasing as a representable-functor statement (Θ represents a certain $Cat_{\mathcal{C}}^{\Sigma\text{-reg}} \to$ Set-valued functor): the two are logically equivalent, and every place this fact is used below invokes it exactly in the "iff" form.

6. Restricting a center

The remainder of the paper studies operations that produce new centers, and new dilatations, from old ones. The simplest is restriction to a subfamily of indices.

Fix a center $Z = \{(d_i, N_i)\}_{i\in I}$ and a nonempty subset $K \subset I$. Restricting Z to K (forgetting every pair with $i \notin K$) gives a new center $Z|_K = \{(d_i, N_i)\}_{i\in K}$ on the same category $\mathcal{C}$, hence a new dilatation $\mathcal{C}[\{(d_i)^{-1} \circ N_i\}_{i\in K}]$. Since every $\{(d_i, N_i)\}_{i\in K}$-fraction is in particular a $\{(d_i, N_i)\}_{i\in I}$-fraction, there is a canonical comparison functor.

Proposition 6.1 ([5, Proposition 3.14])**.** *There is a canonical functor* $\Phi : \mathcal{C}[\{(d_i)^{-1} \circ N_i\}_{i\in K}] \to \mathcal{C}[\{(d_i)^{-1} \circ N_i\}_{i\in I}]$. *Moreover:*

(i) if $N_i = S_{d_i}$ *for every* $i \in I \smallsetminus K$, *then* Φ *is full;*
(ii) if $\mathcal{C}[\Gamma^{-1}] \to \mathcal{C}[\Sigma^{-1}]$ *is faithful, where* $\Gamma = \{d_i\}_{i\in K}$, *then* Φ *is faithful.*

In Lean, `Center.restrict Z K hK` builds $Z|_K$ (reindexing every field of `Center` along the inclusion $K \hookrightarrow I$), and Φ itself, `restrictPhi`, is produced not by hand but as `DilaLift` — the functor F' of Theorem 5.8 (§5) — applied to $F = \Theta_I : \mathcal{C} \to \mathcal{C}[\{(d_i)^{-1} \circ N_i\}_{i\in I}]$ restricted along $Z|_K$: the sieve inequalities of condition (1) for $Z|_K$ are exactly the $i \in K$ instances of the sieve inequalities already known to hold for Θ_I on the whole of Z (Proposition 5.5), and condition (2) is Fact 5.7 for Θ_I, restricted along $K \subset I$ using the generic faithfulness-restriction lemma of §5.2. This is an instructive small example of the "prove the universal property once, specialize it everywhere" pattern: Φ is not an ad hoc functor with its own bespoke construction, it is Theorem 5.8's output for a particular choice of target.

6.1. Fullness: an explicit preimage construction.

Part (i) needs more than the universal property alone: it requires producing, for an arbitrary morphism of $\mathcal{C}[\{(d_i)^{-1} \circ N_i\}_{i\in I}]$, an explicit preimage under Φ. The proof is again organized by Lemma 5.9, now applied inside the target category's generated presentation rather than the source's: every morphism of the big dilatation is (the image of) a path built from original and fraction generators, and a preimage is constructed by induction on this path.

```
def PhiPreimage (Z : Center C) (K : Set Z.I) (hK : K.Nonempty)
A B : GeneratedCategory Z (p :  A ⟶ B) : Prop :=
∃ f :  ..., ...
```

(the precise statement records the existence of a morphism of the small dilatation mapping, under Φ, to the image of p). The three structural cases

of the induction are handled by `PhiPreimage_id` and `PhiPreimage_comp` (trivial: the identity and composition of preimages are preimages of the identity and the composition — the identity case even closes by `rfl`, since `restrictPhi`, being the structural `DilaLift` rather than an opaque existential witness, makes the object identification $\Phi(\Theta_{Z|_K}(X)) = \Theta_I(X)$ definitional, so the `eqToHom` transports flanking the identity reduce to identities), and the generator case is where hypothesis (i) is used, in `PhiPreimage_edge`: an original generator $\Theta_I(a)$ is already in the image of Φ (take $\Theta_K(a)$); and a fraction generator for (i, n) is handled by cases on whether $i \in K$ (again already in the image) or $i \notin K$, in which case hypothesis (i) gives $n = q \circ d_i$ for some q, and the fraction morphism for (i, n) collapses, under Θ_I, to $\Theta_I(q)$, an original generator, hence again manifestly in the image of Φ. Assembling the three cases by induction on the path gives `PhiPreimage_all`, from which fullness of Φ, `restrictPhi_full`, follows by transporting an arbitrary morphism of the quotient back to a path (using that `GeneratedToDila` is full, as in §5.3), applying `PhiPreimage_all`, and pushing the resulting preimage back down.

6.2. **Faithfulness.** Part (ii) is more direct: both dilatations map faithfully into their respective raw localizations (Fact 4.1), and Φ composed with the map to the big raw localization factors through the small raw localization (essentially by construction of Φ via the universal property); hypothesis (ii)'s faithfulness of the comparison between the two raw localizations then gives faithfulness of Φ itself via `faithful_of_comp_faithful`, the same one-line categorical lemma from §5.2 used for [5, Fact 3.7] and, again, below.

7. Shrinking the sieves

A second comparison functor sits alongside Proposition 6.1's Φ: rather than dropping indices from I, one can shrink each sieve N_i to an arbitrary sub-sieve $M_i \subset N_i$, keeping the same family of morphisms $\{d_i\}_{i \in I}$.

Fact 7.1 ([5, Fact 3.13])**.** *Let $Z = \{(d_i, N_i)\}_{i \in I}$ be a center and $M_i \subset N_i$ a sieve over $\mathrm{cod}(d_i)$ for every i; write Z_M for the center $\{(d_i, M_i)\}_{i \in I}$. There is a canonical functor $\varphi : \mathcal{C}[\{(d_i)^{-1} \circ M_i\}_{i \in I}] \to \mathcal{C}'$, and φ is faithful.*

As with Φ in §8.1, φ (`Fact313Phi`) is built directly from Theorem 5.8 applied to $\Theta : \mathcal{C} \to \mathcal{C}'$: Σ-regularity of Θ transports unchanged to Z_M (`IsSigmaRegular_altSieve`, since Z_M shares the same underlying Σ as Z), and the sieve inequality is the subsieve inclusion $M_i \subset N_i$ composed with Proposition 5.5's inequality for Z. Faithfulness is where the shared-Σ observation pays off a second time: since Z_M and Z have the same Σ, they have the same raw localization and the same functor $L =$`LocalizationFunctor`, so $\varphi \ggg$`DilaToLoc` Z and `DilaToLoc` Z_M both factor L through Θ_{Z_M}, so `Dila_factor_unique` identifies them outright, and `DilaToLoc` Z_M is faithful unconditionally (Fact 4.1), so `faithful_of_comp_faithful` gives faithfulness of φ. No new machinery appears anywhere in this proof: it is

Theorem 5.8, `Dila_factor_unique`, and `faithful_of_comp_faithful` once more, the same three tools already doing the load-bearing work throughout §6–§8.

Design choice. The printed proof is one sentence (an M-fraction is already an N-fraction, so φ is the identity on representatives); that does not transport here, since the two dilatations are quotients of different generated categories, so φ is instead built from Theorem 5.8 and `Dila_factor_unique`.

8. Combining two centers

The second, and by far the most delicate, operation on centers combines two centers on the same category into one, and relates the resulting dilatation to an iterated dilatation, first dilate by one center, then dilate the result by (the pushforward of) the other. We present this section in the order the formalization itself builds it: first the construction and the six declarations it produces, entirely in the terms the source file uses, and only afterward do we state the corresponding result of the printed paper ([5, Proposition 3.15]) and compare the two. This reordering is not merely stylistic: for this particular result, the printed statement and the formal one turn out not to say quite the same thing, and the difference is easiest to see after the formal construction is already on the table, not before.

8.1. The construction, in the order the formalization builds it.

Fix two centers $Z = \{(d_i, N_i)\}_{i\in I}$ and $W = \{(d_j, N_j)\}_{j\in J}$ on the same category $\mathcal{C}$.

The combined center. The formalization first forms the center that simply puts Z and W side by side, indexed by the disjoint union $I \sqcup J$:

```
 def Center.sum (Z W : Center C) : Center C where
I := Z.I ⊕ W.I
nonempty := ⟨Sum.inl Z.nonempty.some⟩
dom := Sum.elim Z.dom W.dom
cod := Sum.elim Z.cod W.cod
mor := fun i => match i with | Sum.inl i => Z.mor i | Sum.inr j =>
W.mor j
N := fun i => match i with | Sum.inl i => Z.N i | Sum.inr j => W.N j
```

every field is simply routed to the Z- or the W-side by cases on the summand. Two immediate containment facts about the associated collections of central morphisms, `CenterMorphismProperty_sum_inl_le` and `CenterMorphismProperty_sum_inr_le` (each a two-line proof: an index witnessing $f \in \Sigma_Z$ or $f \in \Sigma_W$ certainly witnesses $f \in \Sigma_{Z\sqcup W}$, via `Sum.inl` or `Sum.inr`), together with their F-image analogues `ImageCenterMorphismProperty_sum_inl_le` and `..._sum_inr_le` for an arbitrary functor F out of $\mathcal{C}$, are recorded once and reused repeatedly below:

they are the only place the definition of `Center.sum` is unfolded directly, everything else works with these two containments as a black box.

Regularity of $\Theta_{Z\sqcup W}$ on each side. Before anything is built, the formalization records that $\Theta_{Z\sqcup W} : \mathcal{C} \to \mathcal{C}_{Z\sqcup W}$ (which is already known to be $\Sigma_{Z\sqcup W}$-regular, as an instance of Fact 5.7 applied to the combined center) remains regular when restricted to either half of the index set:

```
lemma CatToDila_isSigmaRegular_sum_inl (Z W : Center C) :
IsSigmaRegular Z (CatToDila (Z.sum W)) := by
show (ImageCenterMorphismProperty Z (CatToDila (Z.sum
W))).Q.Faithful
apply faithful_of_comp_faithful
(ImageCenterMorphismProperty Z (CatToDila (Z.sum W))).Q
(Localization.Construction.lift ...  (ImageCenterMorphismProperty
(Z.sum W)
(CatToDila (Z.sum W))).Q ...)
rw [Localization.Construction.fac]
exact CatToDila_isSigmaRegular (Z.sum W)
```

with a symmetric statement `CatToDila_isSigmaRegular_sum_inr` for W. Both are instances of the same one-line categorical lemma from §5.2, `faithful_of_comp_faithful`: the localization at the Z-part alone factors, via `Localization.Construction.lift`, through the localization at the full $Z \sqcup W$ family (using the containment `ImageCenterMorphismProperty_sum_inl_le` above to know every morphism the bigger localization needs to invert to make this factoring functor well-defined is already inverted), and the bigger localization is faithful by Fact 5.7. This is the same "sub-family of a regular family is regular" pattern already used in §6, now instantiated for the two halves of a disjoint union instead of for an arbitrary subset $K \subset I$.

β: dilating $\mathcal{C}_Z$ by the pushed-forward W. Pushing W forward along $\Theta_Z : \mathcal{C} \to \mathcal{C}_Z :=$`Dila` Z gives a center on $\mathcal{C}_Z$ itself, and dilating $\mathcal{C}_Z$ by it gives a functor β:

```
def CenterZW (Z W : Center C) : Center (Dila Z) := W.pushforward
(CatToDila Z)
def Beta315 (Z W : Center C) : Dila Z ⇒ Dila (CenterZW Z W) :=
CatToDila (CenterZW Z W)
```

`Center.pushforward` is the general-purpose operation transporting a center along an arbitrary functor (sieve $N_j \mapsto$ its pushforward `Sieve.functorPushforward` (Θ_Z) N_j, already used for Proposition 5.5); β is then, verbatim, Θ for this pushed-forward center, i.e. nothing more than another instance of Definition 3.1–§4's basic construction, applied one level up, inside $\mathcal{C}_Z$ instead of inside $\mathcal{C}$.

Φ: comparing $\mathcal{C}_Z$ directly with $\mathcal{C}_{Z\sqcup W}$. A comparison functor $\Phi : \mathcal{C}_Z \to \mathcal{C}_{Z\sqcup W}$ is produced next, not by invoking Proposition 6.1's `restrictPhi` (which would be the obvious route, $Z \sqcup W$ restricted to its Z-part being Z again) but by a fresh application of Theorem 5.8 to $\Theta_{Z\sqcup W}$ itself:

```
noncomputable def Phi315 (Z W : Center C) : Dila Z ⇒ Dila (Z.sum
W) :=
DilaLift Z (CatToDila (Z.sum W))
(CatToDila_isSigmaRegular_sum_inl Z W) (CatToDila_sum_hsieve_inl Z
W)

lemma Phi315_spec (Z W : Center C) :
CatToDila Z ⋙ Phi315 Z W = CatToDila (Z.sum W) :=
DilaLift_fac Z (CatToDila (Z.sum W))
(CatToDila_isSigmaRegular_sum_inl Z W) (CatToDila_sum_hsieve_inl Z
W)
```

Regularity of Φ.

```
theorem Phi315_isSigmaRegular (Z W : Center C) :
IsSigmaRegular (CenterZW Z W) (Phi315 Z W) := by
show (ImageCenterMorphismProperty (CenterZW Z W) (Phi315 Z
W)).Q.Faithful
rw [ImageCenterMorphismProperty_ZW_Phi_eq]
exact CatToDila_isSigmaRegular_sum_inr Z W
```

The single nontrivial ingredient is `ImageCenterMorphismProperty_ZW_Phi_eq`, an extensionality lemma identifying the images of the `CenterZW`-generators under Φ with the images of the W-generators under $\Theta_{Z\sqcup W}$ directly (using `Phi315_spec` to transport along the object-level identification $\Phi(\Theta_Z(X)) = \Theta_{Z\sqcup W}(X)$, with the `eqToHom` bookkeeping this identification carries, exactly as in §5.3); once the two image-classes of morphisms are known to coincide, regularity is exactly `CatToDila_isSigmaRegular_sum_inr` from above.

The comparison functor in the other direction, built first. The formalization builds a second comparison functor, $\alpha' : \mathcal{C}_Z[\texttt{CenterZW}] \to \mathcal{C}_{Z\sqcup W}$, before it builds the one going the other way, and the source file's own comment explains why: the construction still to come will need this one's defining equation as an ingredient, so it is built first, and it needs nothing beyond the regularity of Φ just established:

```
noncomputable def Alpha'315 (Z W : Center C) : Dila (CenterZW Z W)
⇒ Dila (Z.sum W) :=
DilaLift (CenterZW Z W) (Phi315 Z W)
(Phi315_isSigmaRegular Z W) (Phi315_hsieve Z W)

theorem Alpha'315_spec (Z W : Center C) :
```

```
Beta315 Z W ⋙ Alpha'315 Z W = Phi315 Z W :=
DilaLift_fac (CenterZW Z W) (Phi315 Z W)
(Phi315_isSigmaRegular Z W) (Phi315_hsieve Z W)
```

This is Theorem 5.8 applied a third time in this section alone (after the two applications already used for Φ): to Φ itself, viewed as a functor out of $\mathcal{C}_Z$'s `CenterZW`-dilatation, using exactly the regularity and sieve-inclusion facts already in hand (`Phi315_hsieve`, the pushforward-sieve inequality, is again Proposition 5.5 transported through `Sieve.functorPushforward_comp`, the same pattern as [5, Fact 3.11]).

The delicate direction, and the hypothesis it needs. The comparison functor going the other way, $\alpha : \mathcal{C}_{Z\sqcup W} \to \mathcal{C}_Z[\texttt{CenterZW}]$, is delicate. Its defining application of Theorem 5.8 needs $\Theta_Z \ggg \beta$ to be $\Sigma_{Z\sqcup W}$-regular, which the machinery built so far does not supply automatically; see §8.2. The formalization's response is to stop trying to derive it and instead take it as an explicit hypothesis, threaded through every following declaration by name:

```
 noncomputable def Alpha315 (Z W : Center C)
(hreg :  IsSigmaRegular (Z.sum W) (CatToDila Z ⋙ Beta315 Z W)) :
Dila (Z.sum W) ⇒ Dila (CenterZW Z W) :=
(Dila_universal_property (Z.sum W) (CatToDila Z ⋙ Beta315 Z W)
hreg (BetaComp315_hsieve Z W)).choose
```

where `BetaComp315_hsieve` is the accompanying sieve-inclusion hypothesis.

Assembling the isomorphism. With `hreg` in hand, the remaining declarations are again routine, and again go through Theorem 5.8's uniqueness clause, now via a standalone helper, `Dila_factor_unique`, recording exactly that clause for reuse (§5.3), applied three times in succession. First, an auxiliary identity:

```
 theorem Phi315_comp_Alpha315 (Z W : Center C)
(hreg :  IsSigmaRegular (Z.sum W) (CatToDila Z ⋙ Beta315 Z W)) :
Phi315 Z W ⋙ Alpha315 Z W hreg = Beta315 Z W := by
apply Dila_factor_unique Z (CatToDila Z ⋙ Beta315 Z W)
(Phi315 Z W ⋙ Alpha315 Z W hreg) (Beta315 Z W)
· rw [← Functor.assoc, Phi315_spec, Alpha315_spec]
· rfl
· exact IsSigmaRegular_sum_inl_of Z W (CatToDila Z ⋙ Beta315 Z W)
hreg
```

(both $\Phi \ggg \alpha$ and β agree after precomposing with Θ_Z, so `Dila_factor_unique` (fed the restriction of `hreg` to the Z-part alone, via `IsSigmaRegular_sum_inl_of`, the same restriction-of-regularity fact used throughout §8.1) identifies them). From this single identity, two further

identities follow by the same pattern, cancelling in the other two dilatations in turn:

```
 theorem Alpha315_comp_Alpha'315 (Z W : Center C)
(hreg :  ...)  :  Alpha315 Z W hreg ⋙ Alpha'315 Z W = 𝟙 (Dila
(Z.sum W)) := by
apply Dila_factor_unique (Z.sum W) (CatToDila (Z.sum W)) ...  (𝟙 _)
· rw [← Functor.assoc, Alpha315_spec, Functor.assoc, Alpha'315_spec,
Phi315_spec]
· exact Functor.comp_id _
· exact CatToDila_isSigmaRegular (Z.sum W)

theorem Alpha'315_comp_Alpha315 (Z W : Center C)
(hreg :  ...)  :  Alpha'315 Z W ⋙ Alpha315 Z W hreg = 𝟙 (Dila
(CenterZW Z W)) := by
apply Dila_factor_unique (CenterZW Z W) (Beta315 Z W) ...  (𝟙 _)
· rw [← Functor.assoc, Alpha'315_spec, Phi315_comp_Alpha315]
· exact Functor.comp_id _
· exact CatToDila_isSigmaRegular (CenterZW Z W)
```

and finally, a pair of mutually inverse functors assembles into an isomorphism of `Cat`:

```
 noncomputable def Iso315 (Z W : Center C)
(hreg :  IsSigmaRegular (Z.sum W) (CatToDila Z ⋙ Beta315 Z W)) :
Cat.of (Dila (CenterZW Z W)) ≅ Cat.of (Dila (Z.sum W)) where
hom := Alpha'315 Z W
inv := Alpha315 Z W hreg
hom_inv_id := Alpha'315_comp_Alpha315 Z W hreg
inv_hom_id := Alpha315_comp_Alpha'315 Z W hreg
```

needing only the two composite identities just proved, not the fuller coherence data of a `CategoryTheory.Equivalence`.

8.2. **Comparison with the printed paper.** Having built `Center.sum`, Φ, β, `Alpha'315`, `Alpha315`, and `Iso315` in the order above, we can now state what the original paper claims these declarations formalize, and compare.

The statement to be examined is [5, Proposition 3.15]:

> With $\mathcal{C}_Z := \mathcal{C}[\{(d_i)^{-1} \circ N_i\}_{i\in I}]$, $\mathcal{C}_{I'} := \mathcal{C}[\{(d_k)^{-1} \circ N_k\}_{k\in I'}]$ for $I' = I \sqcup J$, and $\mathcal{C}_Z[\{\Theta_Z(d_j)^{-1} \circ \Theta_Z(N_j)\}_{j\in J}]$ the dilatation of $\mathcal{C}_Z$ by the pushed-forward W:
>
> (1) Φ is $\{\Theta_Z(d_j)\}_{j\in J}$-regular;
> (2) $\Theta_Z \ggg \beta$ is $\{d_k\}_{k\in I'}$-regular;
> (3) there is a unique $\alpha : \mathcal{C}_{I'} \to \mathcal{C}_Z[\{\Theta_Z(d_j)^{-1} \circ \Theta_Z(N_j)\}_{j\in J}]$ with $\Theta_Z \ggg \beta = \Theta_{I'} \ggg \alpha$;
> (4) there is a unique $\alpha' : \mathcal{C}_Z[\{\Theta_Z(d_j)^{-1} \circ \Theta_Z(N_j)\}_{j\in J}] \to \mathcal{C}_{I'}$ with $\Phi = \beta \ggg \alpha'$;

(5) $\alpha \ggg \alpha' = \mathrm{id}$ and $\alpha' \ggg \alpha = \mathrm{id}$;
(6) hence $\mathcal{C}_Z[\{\Theta_Z(d_j)^{-1} \circ \Theta_Z(N_j)\}_{j\in J}] \cong \mathcal{C}_{I'}$.

As shown below, part (2)'s printed proof is not rigorous and is not formalized unconditionally here; parts (3), (5), (6), which are built on it, are correspondingly not rigorously established unconditionally as printed either.

Informally, this says: dilating twice, first by Z and then by (the pushforward of) W, gives the same category as dilating once by the union of Z and W, an associativity-of-dilatation statement, and the fact that lets one build up a dilatation by a large, complicated center incrementally, one piece at a time.

Parts (1) and (4) match the construction of §8.1 exactly and unconditionally: (1) is `Phi315_isSigmaRegular`; (4) is `Alpha'315` together with `Alpha'315_spec` and `Alpha'315_unique`. Parts (3), (5), (6) match the construction too, but only conditionally, since α itself already needs part (2)'s regularity to be built at all: (3) is `Alpha315` together with `Alpha315_spec` and `Alpha315_unique`, each taking `hreg` as an explicit argument; (5) is `Alpha315_comp_Alpha'315` and `Alpha'315_comp_Alpha315`, both stated for an arbitrary `hreg`; (6) is `Iso315`, likewise. In other words, item (2) is not merely "the part where the two accounts part ways" in isolation; it is a hypothesis that then propagates through every part built on top of α, i.e. through (3), (5), and (6) as well.

Part (2) of [5, Proposition 3.15] is not fully justified. The printed proof says applying [5, Fact 2.14] twice suffices to prove part (2); that sufficiency is not clear, and no justification for it appears in the source material. We do not know whether part (2) itself, as stated in [5], is true in full generality, only that this proof does not establish it in full generality.

Remark 8.1. Concretely, the formalization proves parts (1) and (4) unconditionally, but parts (2), (3), (5), (6) only conditionally on the extra hypothesis `hreg`.

8.3. **A parallel construction without the gap: combining sieves.** A close relative of the construction above combines, on the same family of morphisms $\{d_i\}_{i\in I}$, two different choices of sieves rather than two different centers, and here the formalization proves the analogous regularity unconditionally.

The Lean construction. For each $i \in I$, let N_i' be another sieve over $\mathrm{cod}(d_i)$, and $N_i'' := N_i \cup N_i'$ their union (`Center.sieveUnion`); let `Center.altSieve` denote the center on $\mathcal{C}_Z$ obtained by pushing forward N_i' (rather than N_i) along Θ_Z. The two comparison functors are built exactly as `Alpha315` and `Alpha'315` were, via Theorem 5.8, applied once in each direction, and recorded as `Alpha318` and `Alpha'318`, with `Iso318` assembling them into an isomorphism of categories, mirroring `Iso315` verbatim.

Comparison with the printed paper.

Proposition 8.2 ([5, Proposition 3.18])**.** *With notation as above,* $\mathcal{C}[\{(d_i)^{-1} \circ N_i\}_{i\in I}, \{(d_i)^{-1} \circ N_i'\}_{i\in I}]$ *(dilating once by* N_i*, then again by the pushforward of* N_i'*) is canonically isomorphic to* $\mathcal{C}[\{(d_i)^{-1} \circ N_i''\}_{i\in I}]$*.*

Unlike [5, Proposition 3.15], every part of this statement (including the analogue of part (2)) is proved unconditionally in the formalization, with no `hreg`-style hypothesis anywhere in `Alpha318`, `Alpha'318`, or `Iso318`. The reason is structural, and it isolates precisely what made §8.2 delicate: here both sides invert exactly the same collection of morphisms $\{d_i\}_{i\in I}$, only the sieves differ, so the problematic step of §8.2, comparing localizations at two different, growing collections of inverted morphisms, never arises; the relevant regularity is inherited directly from Fact 5.7 with no enlargement of Σ to account for. This confirms that the difficulty isolated in Remark 8.1 is specifically about enlarging the family of inverted morphisms, not about combining two pieces of data on $\mathcal{C}$ as such.

9. Codilatations

Everything so far dilates along morphisms $d_i : \mathrm{dom}(d_i) \to \mathrm{cod}(d_i)$ using sieves N_i over $\mathrm{cod}(d_i)$ (collections closed under precomposition), forcing morphisms into $\mathrm{cod}(d_i)$ to factor through d_i. There is an evidently dual construction, forcing morphisms out of $\mathrm{dom}(d_i)$ to factor through d_i, using cosieves (collections of morphisms out of a fixed object, closed under postcomposition) in place of sieves.

Rather than developing this dual theory from scratch (repeating every definition, lemma, and proof of §3–§5 with arrows reversed), the formalization derives it for free, by definitional transport across the opposite-category duality $\mathcal{C} \leftrightsquigarrow \mathcal{C}^{\mathrm{op}}$.

Fact 9.1. *A cosieve from an object* X *of* $\mathcal{C}$ *is the same data as a sieve over* X*, regarded as an object of* $\mathcal{C}^{\mathrm{op}}$*.*

In Lean this identification is recorded as an explicit `Equiv` (`Cosieve.equivSieveOp`), not merely as a pair of maps going back and forth: a `Cosieve X` is a structure of identical shape to `Sieve`, and the equivalence simply reindexes the closure condition along op.

Definition 9.2 ([5, Definition 4.1])**.** A cocenter in $\mathcal{C}$ is a family $\{(d_i, V_i)\}_{i\in I}$ with d_i a morphism and V_i a cosieve from $\mathrm{dom}(d_i)$.

Given the identification above, a cocenter in $\mathcal{C}$ is a center in $\mathcal{C}^{\mathrm{op}}$ (with d_i replaced by its formal opposite $d_i^{\mathrm{op}} : \mathrm{cod}(d_i) \to \mathrm{dom}(d_i)$, now going the right way to serve as a center morphism in $\mathcal{C}^{\mathrm{op}}$); Lean's `Cocenter.toCenterOp` records exactly this re-casting, and no new `structure` for "cocenter data attached to $\mathcal{C}$ itself" beyond the re-casting is introduced.

Definition 9.3 ([5, Definition 4.3])**.** The codilatation of $\mathcal{C}$ with cocenter $\{(d_i, V_i)\}_{i\in I}$ is $\mathcal{C}[\{V_i \circ (d_i)^{-1}\}_{i\in I}] := \big(\mathcal{C}^{\mathrm{op}}[\{(d_i)^{-1} \circ V_i\}_{i\in I}]\big)^{\mathrm{op}}$.

```
def Codila (co :  Cocenter C) : Type u := (Dila (co.toCenterOp))ᵒᵖ
```

This one line is the whole construction: dilate $\mathcal{C}^{\mathrm{op}}$ by the corresponding center, then take the opposite category again to land back in (a category built from) $\mathcal{C}$. Every structural fact then transports along the same route, with no new proof content:

Proposition 9.4 ([5, Proposition 4.5])**.** *There is a canonical functor* $\Upsilon : \mathcal{C} \to \mathcal{C}[\{V_i \circ (d_i)^{-1}\}_{i\in I}]$*, a canonical faithful functor from the codilatation to* $\mathcal{C}[\{d_i\}_{i\in I}^{-1}]$*,* Υ *is* $\{d_i\}_{i\in I}$*-regular, and* Υ *represents the appropriately dualized functor* $Cat_{\mathcal{C}}^{\{d_i\}_{i\in I}\text{-}reg} \to \mathrm{Set}$*.*

In Lean, Υ is `Cocenter.Upsilon co`, built as `(CatToDila co.toCenterOp).rightOp` (Mathlib's operation turning a functor $\mathcal{C}^{\mathrm{op}} \to \mathcal{D}$ into a functor $\mathcal{C} \to \mathcal{D}^{\mathrm{op}}$); the faithful comparison functor is `Codila.faithful_to_loc_op`, an instance derived directly from Fact 4.1 applied in $\mathcal{C}^{\mathrm{op}}$; Σ-regularity of Υ is `Cocenter.Upsilon_isSigmaRegular`, from Fact 5.7 transported across op (using, once more, `faithful_of_comp_faithful` to relate faithfulness of a functor and of its `.op`); and representability is `Cocenter.represents`, from [5, Proposition 3.12] transported across op, combined with the sieve/cosieve identification above. Not a single one of these four proofs contains an argument that is not already present, in dual form, in §5.1–§5.3; the work is entirely in setting up the op-translation once, correctly, and then invoking it four times.

10. Comparison with dilatations of rings

The construction that motivated this whole theory, dilatations of a commutative ring A at a multi-center $\{(M_i, a_i)\}_{i\in I}$ of ideals and elements, should be a special case of dilatations of categories. We present the formalization's own construction first, in the order the source builds it, and only afterward state the printed paper's Proposition 5.1 and compare the two: as with [5, Proposition 3.15] above, the comparison is easiest to see once the construction is already on the table.

10.1. The construction, following the formalization.

The vendored ring dilatation. The ring $A[M]$ itself is not developed inside the file discussed in this paper: it is vendored, essentially verbatim, from the Lean formalization of multi-centered dilatations of rings [8], accompanying the separate paper introducing the construction mathematically [4]. A multi-center is a triple of an index type and two functions:

```
structure Multicenter (A' : Type*) [CommSemiring A'] where
index :  Type*
ideal :  index → Ideal A'
```

```
elem :  index → A'
```

Writing $L_i := M_i + (a_i)$ for the associated *large ideal* (`LargeIdeal`), and, for a finitely-supported exponent tuple $\nu \in \mathbb{N}^{(I)}$ (Lean notation $M^{\mathbb{N}}$ for `index` $\to_0$ $\mathbb{N}$), $a^\nu := \prod_i a_i^{\nu_i}$ and $L^\nu := \prod_i L_i^{\nu_i}$ (`familyPow`, `prodLargeIdealPower`), a general element of $A[M]$ is a pair (ν, m) with $m \in L^\nu$ (`PreDil`), modulo the equivalence $(\nu, m) \sim (\nu', m')$ iff $m \cdot a^{\nu'+\beta} = m' \cdot a^{\nu+\beta}$ for some $\beta \in \mathbb{N}^{(I)}$ (`Multicenter.r`); $A[M]$ (`Multicenter.Dilatation`) is the resulting quotient. This is a deliberate scope decision: re-deriving the ring-theoretic theory here would duplicate an existing, independently-developed formalization, and the only new content needed for this paper is the comparison with dilatations of categories, described next.

The index type $M^{\mathbb{N}}$.

```
def centerOfMulticenter (M : Multicenter A') :
Center (CategoryTheory.SingleObj A') where
I := M^ℕ
nonempty := ⟨0⟩
dom _ := CategoryTheory.SingleObj.star A'
cod _ := CategoryTheory.SingleObj.star A'
mor ν := M.elem^ν
N ν := Sieve.ofIdeal (M.LargeIdeal^ν)
```

The index type of this center is $M^{\mathbb{N}} = \mathbb{N}^{(I)}$, the same exponent profiles indexing $A[M]$'s elements above — *not* M.index $= I$: the generator at ν divides by a^ν, with numerator ranging over all of L^ν.

The universal-property half. Feeding the canonical ring map $A \to A[M]$ through `SingleObj` gives `toDilatationFunctor`:`SingleObj` $A \to$`SingleObj` $A[M]$, the functor playing the role of F in Theorem 5.8. `prop_5_1` applies that theorem to it directly: the sieve inequality is a direct computation (an element of the image of L^ν already factors through a^ν, by definition of L^ν), and Σ-regularity is built by composing with the further localization of $A[M]$ at the submonoid generated by the images of the a_i's — faithful because those images are non-zero-divisors in $A[M]$ (`nonzerodiv_image_single`, an unconditional structural fact about the vendored construction), so this further localization is injective. This gives

```
noncomputable def Phi51 :  Dila (centerOfMulticenter M) ⇒
CategoryTheory.SingleObj A'[M] :=
(prop_5_1 M).choose
```

together with `Phi51_spec` and `Phi51_unique`, exactly as `Alpha315` was extracted from `Dila_universal_property` in §8.1.

Faithfulness, via a commutativity argument. Theorem 5.8 only guarantees Φ =`Phi51` exists; showing it is an isomorphism is separate work. Faithfulness goes through a second comparison functor, `kappaFunctor`:`SingleObj` $A[M]$ $\to$ the raw localization of `centerOfMulticenter` M, built from a monoid map `LocEndLift` extending `toLocEnd`: A $\to$`End`$(\bullet)$ along the localization $A \to A[M]$. Building `LocEndLift` needs the target monoid to be commutative, which is not automatic — it is established by `allCentral`: every endomorphism of the raw localization's single object commutes with every other morphism, proved by showing the relevant `MorphismProperty` is stable under composition, contains every generator-image (central because A is commutative, `genImage_central`), and contains every formal inverse (a central element's inverse is central), hence is everything (`Localization.Construction.morphismProperty_is_top`). With commutativity in hand, `Phi51`$\ggg$`kappaFunctor` is identified with the (unconditionally faithful, Fact 2.14) `DilaToLoc` of `centerOfMulticenter` M by `Dila_factor_unique`, and `faithful_of_comp_faithful` gives `Phi51_faithful`.

Fullness, for free from the indexing. Fullness needs none of this machinery. Since the center is indexed by exponent profiles, every element of $A[M]$ is *directly* a pair (ν, m) by `Multicenter.Dilatation.induction_on`, hence directly the image of the single fraction generator at (ν, m), using the defining identity $a^\nu \cdot (m/a^\nu) = m$ inside $A[M]$ itself (`algebraMap_elem_pow_mul_frac`). No induction on composites is needed, unlike the fullness arguments of §6 and §11.

Assembling the isomorphism. With `Phi51` full and faithful, an explicit inverse `Psi51` is built directly (rather than invoked from a general full-faithful-essentially-surjective equivalence principle, since both categories here have a single object, so Φ is precisely a bijective monoid homomorphism and its set-theoretic inverse is again one), and

```
noncomputable def Iso51 :  Cat.of (Dila (centerOfMulticenter M)) ≅
Cat.of (CategoryTheory.SingleObj A'[M]) where
hom := Phi51 M
inv := Psi51 M
hom_inv_id := Phi51_comp_Psi51 M
inv_hom_id := Psi51_comp_Phi51 M
```

assembles the two composite-identity theorems `Phi51_comp_Psi51`/`Psi51_comp_Phi51` into an isomorphism of categories, matching the pattern of `Iso315` in §8.1 exactly.

The identification above assembles into a single chain together with one further fact, purely ring-theoretic and proved independently of categories: reindexing the multi-center by exponent profiles and dilating again recovers the same ring.

Theorem 10.1. *Let $\{[M_i, a_i]\}_{i\in I}$ be a multi-center on A, with large ideals $L_i = M_i + (a_i)$, and for $\nu \in \mathbb{N}^{(I)}$ write $L^\nu := \prod_i L_i^{\nu_i}$, $a^\nu := \prod_i a_i^{\nu_i}$. Then*

$$\mathcal{C}\big[\{(a^\nu)^{-1} \circ L^\nu\}_{\nu\in\mathbb{N}^{(I)}}\big] \;\cong\; A\big[\{L^\nu/a^\nu\}_{\nu\in\mathbb{N}^{(I)}}\big] \;\cong\; A\big[\{M_i/a_i\}_{i\in I}\big] = A[M],$$

where $\mathcal{C}$ is the one-object category attached to A.

The first isomorphism is `Iso51` above: the categorical dilatation by the ν-indexed center `centerOfMulticenter` M is isomorphic to $A[M]$ itself. The second is `Multicenter.Dilatation.reindexRingEquiv`: dilating A by the ν-indexed multi-center $\{[L^\nu, a^\nu]\}_{\nu\in\mathbb{N}^{(I)}}$ (`Multicenter.reindex` M) recovers $A[M]$ up to isomorphism, via the flattening map $\mu \mapsto \sum_\nu \mu(\nu)\cdot\nu$ sending the reindexed ring's own exponent profiles back down to M's. Together, the two say that all three constructions — the category built from ν-indexed generators, the original ring dilatation, and the ring dilatation of the ν-indexed reindexing — coincide.

10.2. **Comparison with the printed paper.** With the multi-center $\{[M_i, a_i]\}_{i\in I}$ as above, [5, Proposition 5.1] asserts an identification, as $\mathcal{C}$-categories, of $\mathcal{C}[\{(a_i)^{-1} \circ M_i\}_{i\in I}]$ (dilating by the center indexed directly by $i \in I$) with the category attached to $A[M]$. This identification does not hold.

The printed proof's comparison functor Φ sends a fraction generator to m/a_i and is claimed surjective. It is not: composition in `SingleObj` A is multiplication, so the image of every morphism is a product $c \cdot \prod_j (m_j/a_{i_j})$, and not every element of $A[M]$ has this multiplicative form — for instance, with $A = \mathbb{Z}[X]$, $a = 2$, $M = (X)$, every product of elements of M stays divisible by X, but $(X+2)/2 \in A[M]$ does not. Moreover no $\mathcal{C}$-functor can fix this: `toDilatationFunctor` satisfies both hypotheses of Theorem 5.8 for the I-indexed center too, so by uniqueness Φ is the *only* candidate, and it is not an isomorphism.

This argument is formalized in full: `centerNaive` is the naive I-indexed center, `PhiNaive` the comparison functor built from Theorem 5.8's existence clause, `target_elt_not_in_range` proves $(X+2)/2$ is unreachable, and `no_C_compatible_equiv` concludes that no functor compatible with the two canonical inclusion functors is an equivalence.

The structural reason is additive closure: $A[M]$'s numerators range over L^ν, a sum of products of single-ideal elements, but composition alone only ever produces products. The formalization's `centerOfMulticenter` (§10.1) repairs this by moving the needed sums into the sieves themselves (L^ν as a single sieve, rather than M_i generator by generator); with that center, the identification holds (Theorem 10.1 above).

11. A characterization that fails for categories

This section formalizes [5, Fact 5.2], as Theorem 11.1.

11.1. **The ring-theoretic fact being tested.** For dilatations of commutative rings, every sub-A-algebra of a localization $A[(\{a_i\}_{i\in I})^{-1}]$ arises as $A[\{M_i/a_i\}]$ for a suitable multi-center. Theorem 11.1 shows, by an explicit finite counterexample, that the analogous statement fails for dilatations of categories.

11.2. **The counterexample category.** Let $\mathcal{C}$ have two objects X, Y, with $\mathrm{Hom}(X,X) = \{\mathrm{id}_X\}$, $\mathrm{Hom}(Y,Y) = \{\mathrm{id}_Y\}$, $\mathrm{Hom}(Y,X) = \varnothing$, and $\mathrm{Hom}(X,Y) = \{a, b\}$ two distinct, non-identity, parallel morphisms; let $\Gamma = \{b\}$.

```
inductive Obj :  Type
| X | Y

inductive CHom :  Obj → Obj → Type
| idX : CHom .X .X
| idY : CHom .Y .Y
| a :  CHom .X .Y
| b :  CHom .X .Y
```

Composition is a four-line pattern match, and associativity holds by `cases ... <;> rfl` over the finitely many cases.

Localizing at $\Gamma = \{b\}$ produces infinitely many morphisms $X \to Y$: $b, a, ab^{-1}a, \ldots$ In Lean, the pairwise distinctness of the first three is proved with a separating invariant: a functor `FSep` to the one-object groupoid on $(\mathbb{Z}, +)$ (Mathlib's `SingleObj (Multiplicative ℤ)`), sending $a \mapsto 1$ and $b \mapsto 0$. The target being a groupoid, `FSep` inverts Γ trivially, hence extends to $\mathcal{C}[\Gamma^{-1}]$ by the universal property of localization (§2); the resulting integer is additive along composition and takes the values $0, 1, 2$ on $b, a, ab^{-1}a$ (`pairwise_distinct`).

Now let $\mathcal{D}$ be the subcategory of $\mathcal{C}[\Gamma^{-1}]$ with the same objects, with $\mathrm{Hom}_{\mathcal{D}}(Y,X) = \varnothing$, $\mathrm{Hom}_{\mathcal{D}}(X,X) = \{\mathrm{id}_X\}$, $\mathrm{Hom}_{\mathcal{D}}(Y,Y) = \{\mathrm{id}_Y\}$, and $\mathrm{Hom}_{\mathcal{D}}(X,Y) = \{b, a, ab^{-1}a\}$. The evident functors $\mathcal{C} \to \mathcal{D}$ and $\mathcal{D} \to \mathcal{C}[\Gamma^{-1}]$ (the inclusion) compose to the localization functor.

Theorem 11.1 ([5, Fact 5.2])**.** *$\mathcal{D} \to \mathcal{C}[\Gamma^{-1}]$ is faithful, but $\mathcal{D}$ is not isomorphic, as a $\mathcal{C}$-category, to $\mathcal{C}[\{(d_i)^{-1} \circ N_i\}_{i\in I}]$ for any center $\{(d_i, N_i)\}_{i\in I}$ on $\mathcal{C}$.*

Faithfulness of $\mathcal{D} \to \mathcal{C}[\Gamma^{-1}]$ is immediate, $\mathcal{D}$ being literally a subcategory. The content is the second half.

11.3. **The case analysis.** The proof considers an arbitrary center $\{(d_i, N_i)\}_{i\in I}$ on $\mathcal{C}$, writes $\Sigma = \{d_i\}_{i\in I}$, and distinguishes two cases: either (i) $a \in \Sigma$ with $\mathrm{id}_Y \in N_a$ or $b \in N_a$ (or symmetrically for b), in which case $\#\,\mathrm{Hom}_{\mathcal{C}'}(X,Y)$ is infinite; or (ii) whenever $a \in \Sigma$ then $N_a \subset \{a\}$, and whenever $b \in \Sigma$ then $N_b \subset \{b\}$, in which case $\mathcal{C}' = \mathcal{C}$. In both cases $\mathcal{C}' \neq \mathcal{D}$. In Lean, the dichotomy is expressed by a single predicate, with case (i) as its negation.

```
def GoodCenter (Z : Center Obj) : Prop :=
∀ (i : Z.I) (X' : Obj) (m : X' ⟶ Z.cod i), Z.N i m → ∃ q : X'
⟶ Z.dom i, m = q ≫ Z.mor i
```

That is: Z is good if every sieve witness factors through its own generator; a case analysis on the finitely many shapes of generators (`Center.dom_cod_cases`) shows this is equivalent, for a center on $\mathcal{C}$, to case (ii).

11.3.1. *If Z is not good.* Suppose some witness $m \in N_i$ does not factor through d_i. Case-splitting on the shape of d_i (`Center.dom_cod_cases`):

- if $d_i : X \to X$ or $d_i : Y \to Y$, then $d_i = \mathrm{id}$, through which everything factors, contradicting the choice of m (`false_of_hnq_selfmor`);
- if $d_i : X \to Y$, a further case split on m shows that the fraction morphism of Proposition 5.2 for (i, m) is a non-identity endomorphism of X or a morphism $Y \to X$ in $\mathcal{C}'$, and $\mathcal{D}$ has neither (`false_of_hnq_case3`).

In Lean, case (i)'s obstruction is thus exhibited as a single explicit morphism that cannot exist in $\mathcal{D}$ under any isomorphism $\mathcal{C}' \cong \mathcal{D}$ compatible with the maps from $\mathcal{C}$.

11.3.2. *If Z is good.* Suppose Z is good. Lemma 5.9 then shows every morphism of $\mathcal{C}'$ is Θ of a morphism of $\mathcal{C}$: on a fraction generator (i, n), goodness gives $n = q \circ d_i$, and cancelling the epimorphism $\Theta(d_i)$ ([5, Proposition 3.3]) in the defining triangle forces $b = \Theta(q)$. This gives `CatToDila_full_of_good`: Θ is full, and faithful here, so $\mathcal{C}' \cong \mathcal{C}$ as $\mathcal{C}$-categories; since $\mathcal{C}$ has two morphisms $X \to Y$ and $\mathcal{D}$ has three, $\mathcal{C}' \not\cong \mathcal{D}$.

11.4. **Assembling the theorem.** `theorem no_realizing_center :`
`¬ ∃ (Z : Center Obj) (e : DObj ≌ Dila Z), CtoD ⋙ e.functor =`
`CatToDila Z`

Given a hypothetical center Z and equivalence e realizing $\mathcal{D}$ as `Dila` Z compatibly with the maps from $\mathcal{C}$, `no_realizing_center` case-splits on `GoodCenter`: if Z is not good, the impossible morphism of the first case transports across e to a contradiction; if Z is good, the second case produces an injective comparison $\psi : \mathrm{Hom}_{\mathcal{D}}(X, Y) \to \mathrm{Hom}_{\mathcal{C}'}(X, Y)$ under which the three generators b, a, c of $\mathcal{D}$ (with c standing for $ab^{-1}a$) all land in the two-element set $\{\Theta(a), \Theta(b)\}$, forcing a collision that contradicts `pairwise_distinct`.

Appendix A. Dictionary between the mathematics and the Lean formalization

This appendix tabulates, for each numbered statement of the original exposition of dilatations of categories, the Lean declaration(s) that play its role (§A.1–§A.5); the few statements that are not formalized are marked "—", with the reason in the Comments column.

A.1. **Notation dictionary.**

Mathematics	Lean	Comments
$\mathcal{C}$, $\mathcal{D}$	`C`, `D`	`C : Type u [Category.v C]`.
$X \xrightarrow{a} Y$	`a : X ⟶ Y`	$\longrightarrow$ is Mathlib's hom-notation.
$g \circ f$	`f ≫ g`	diagrammatic order, see §1.4.
id_X	`𝟙 X`	
sieve over X	`Sieve X`	Mathlib type, reused as is.
cosieve from X	`Cosieve X`	introduced for this project, §A.4.
$S^{\mathcal{C}}_E$	`Sieve.generate` (applied to a `Presieve`)	e.g. `Sieve.generate (Presieve.singleton g)` for $E = \{g\}$.
$CoS^{\mathcal{C}}_E$	`Cosieve.generate`	
$\{[N_i, d_i]\}_{i\in I}$ ([5, Definition 2.9])	`Center C`	structure, §A.2.
$\{[V_i, d_i]\}_{i\in I}$	`Cocenter C`	structure, §A.4.
$\mathcal{C}[\Sigma^{-1}]$	`(Center MorphismProperty Z).Localization`	Mathlib's `MorphismProperty.Localization`.
$L : \mathcal{C} \to \mathcal{C}[\Sigma^{-1}]$	`LocalizationFunctor Z`	`= (CenterMorphismProperty Z).Q`.
$\mathcal{C}[\{(d_i)^{-1} \circ N_i\}_{i\in I}]$	`Dila Z`	§A.2.
$\mathcal{C}[\{V_i \circ (d_i)^{-1}\}_{i\in I}]$	`Codila co`	§A.4.
$\Theta : \mathcal{C} \to \mathcal{C}'$	`CatToDila Z`	
$\Upsilon : \mathcal{C} \to \mathcal{C}[\{V_i \circ (d_i)^{-1}\}]$	`Cocenter.Upsilon co`	
F' ([5, Theorem 3.10])	`DilaLift Z F hfaith hsieve`	the unique functor with $F' \circ \Theta = F$; defining equation `DilaLift_fac`, uniqueness `DilaLift_unique` (§5).
$b = d_i \backslash n = (d_i)^{-1} \circ n$	`fraction_in_dila_single Z ⟨i, ⟨_, ⟨n, hn⟩⟩⟩`	the witness of Proposition 5.2.

Mathematics	Lean	Comments
$n \circ \ell_{d_i}$ (representative of b)	`fraction_in_path _single Z p`, `fraction_in_loc _single Z p`	the path in `Paths (LocQuiver Σ)`, resp. its class in $\mathcal{C}[\Sigma^{-1}]$; `inv_in_path Z p` is the ℓ_{d_i} factor (Mathlib's ψ_2), see §4.2.
$Cat_{\mathcal{C}}^{\Sigma\text{-reg}}$, membership	`IsSigmaRegular Z F`	a `Prop`, not a category; see §5.2.
$S^{\mathcal{C}'}_{\Theta(N_i)}$, $S^{\mathcal{C}'}_{\Theta(d_i)}$	`CatToDilaSieve Z (Z.N i)`, `Sieve.generate (Presieve.singleton ((CatToDila Z).map (Z.mor i)))`	[5, Definition 3.4] is not given its own name; the two sieves are written out at each use.
$A[\{M_i/a_i\}_{i\in I}]$ (ring dilatation)	`A'[M]` (notation for `Multicenter.Dilatation`)	§A.5.
bimorphism	`Mono f ∧ Epi f`	no dedicated "Bimorphism" class; spelled out.

A.2. **Section 2: sieves, localizations, centers, and dilatations.** As explained in §2, Definitions 2.1–2.8 (sieves, the graph, Σ-sequences and their equivalence, Σ-fractions, the localization, and its universal property) are formalized by reusing Mathlib's calculus-of-fractions localization rather than by reformalizing the graph-theoretic argument of §2 verbatim; the table below records, for each paper statement, which Mathlib declaration plays its role. From [5, Definition 2.9] on, every declaration listed is specific to this project.

Mathematics	Lean declaration	Comments
[5, Definition 2.1] (sieve, $S_E^{\mathcal{C}}$)	`Sieve`, `Sieve.generate`, `Cosieve`, `Cosieve.generate`	Mathlib's `Sieve`; `Cosieve` is introduced for this project (§A.4) since Mathlib has no cosieve type.
[5, Definition 2.2] (graph $\mathcal{G}$)	`Localization.Construction.LocQuiver`	Mathlib; one edge a per morphism of $\mathcal{C}$, one edge l_d per $d \in \Sigma$.
[5, Definition 2.3] (Σ-sequence)	`Quiver.Path`, `Paths`	Mathlib; paths in the `LocQuiver`.

Mathematics	Lean declaration	Comments
[5, Definition 2.4] (equivalence of sequences)	`Localization.Construction.relations`	Mathlib; a congruence on `Paths (LocQuiver Σ)`.
[5, Definition 2.5] (Σ-fraction)	`MorphismProperty.Localization`	Mathlib; underlying type of `(CenterMorphismProperty Z).Localization`.
[5, Fact 2.6] (composite of $l_{d'}, l_d$)	—	no separate Lean counterpart; a special case of the relations of [5, Definition 2.4]/`Localization.Construction.relations`, not restated on its own.
[5, Definition 2.7] ($\mathcal{C}[\Sigma^{-1}]$, L)	`CenterLocalization`, `LocalizationFunctor`	thin wrappers around `(CenterMorphismProperty Z).Localization` / `.Q`.
[5, Proposition 2.8] (universal property of $\mathcal{C}[\Sigma^{-1}]$)	`Localization.Construction.lift`, `Localization.Construction.fac`, `Localization.Construction.uniq`	Mathlib; recovered as an instance of [5, Theorem 3.10] in `Center.ofMorphisms_universal_property`, see §A.5.
[5, Definition 2.9] (center)	`Center`	structure with fields `I`, `nonempty`, `dom`, `cod`, `mor`, `N`; see §3.
[5, Definition 2.10] ($\{[N_i, d_i]\}$-fraction)	`CenterSievePair`, `inv_in_path`, `fraction_in_path_single`, `fraction_in_loc_single`, `IsPairMor`, `PairMorWitness`, `GeneratorMorphismData.fraction`	`CenterSievePair Z := Σ i : Z.I, Σ X : C, f : X ⟶ Z.cod i // Z.N i f` records one (i, n)-generator; its underlying morphism is built in three steps — `inv_in_path` (the edge ℓ_{d_i}, Mathlib's ψ_2), `fraction_in_path_single` (the path-level composite $n \circ \ell_{d_i}$), `fraction_in_loc_single` (its class in $\mathcal{C}[\Sigma^{-1}]$) — and `GeneratorMorphismData` then builds the quiver of §4.

Mathematics	Lean declaration	Comments
[5, Fact 2.11] (associativity of fraction composition)	`Paths.categoryPaths`, `CategoryTheory.Quotient.category`	associativity is inherited for free from Mathlib's path-category and quotient-category instances rather than checked by hand on explicit representative sequences.
[5, Remark 2.12]	`GeneratorMorphismData.original`	the two clauses of the remark are definitional consequences of how `GeneratorQuiver` is built, not separate lemmas.
[5, Definition 2.13] ($\mathcal{C}' = \mathcal{C}[\{(d_i)^{-1} \circ N_i\}]$)	`Dila`, `DilaRel`, `GeneratedCategory`, `GeneratedToDila`	$\mathcal{C}'$ `:= Quotient (DilaRel Z)`; see §4.
[5, Fact 2.14] (faithful functor $\mathcal{C}' \to \mathcal{C}[\Sigma^{-1}]$)	`DilaToLoc`, `Fact_2_14`	`Fact_2_14 : (DilaToLoc Z).Faithful`.
[5, Fact 2.15] ($\mathcal{C}' \cong \mathcal{C}[\Sigma^{-1}]$ when all $N_i = S^{\mathcal{C}}_{Id_{cod(d_i)}}$)	`Center.ofMorphisms`, `CatToDila_ofMorphisms_isIso`, `Fact_2_15`	`Center.ofMorphisms` builds the center with `N := fun _ => ⊤`; `Fact_2_15` is the isomorphism `Cat.of (Dila _) ≅ Cat.of (... .Localization)`.
[5, Corollary 2.16] (smallness)	—	not formalized; size discipline is handled by `Center C`'s universe annotations rather than an explicit smallness lemma.

A.3. Section 3: the universal property and related results.

Mathematics	Lean declaration	Comments
[5, Proposition 3.1(i)] ($\Theta : \mathcal{C} \to \mathcal{C}'$)	`CatToDila`	

Mathematics	Lean declaration	Comments
[5, Proposition 3.1(ii)] (fraction $b = d_i \backslash n$)	`fraction_in_dila_single`, `fraction_in_dila_comp_mor`	existence and the defining triangle; uniqueness is folded into `Dila_universal_property` applied later rather than proved again on the spot.
[5, Fact 3.2] (bimorphism under a faithful functor)	`Fact_3_2`	stated for arbitrary `F : A ⇒ B` with `[F.Faithful]`, not specialized to Θ.
[5, Proposition 3.3] ($\Theta(d_i)$ is a bimorphism)	`Prop_3_3`	
[5, Definition 3.4] ($S^{\mathcal{C}'}_{\Theta(N_i)}$, $S^{\mathcal{C}'}_{\Theta(d_i)}$)	`CatToDilaSieve`	`CatToDilaSieve Z (Z.N i) := Sieve.functorPushforward (CatToDila Z) (Z.N i)`; $S^{\mathcal{C}'}_{\Theta(d_i)}$ itself is not named and is written out as `Sieve.generate (Presieve.singleton ((CatToDila Z).map (Z.mor i)))` at each use.
[5, Proposition 3.5] ($S^{\mathcal{C}'}_{\Theta(N_i)} \subset S^{\mathcal{C}'}_{\Theta(d_i)}$)	`CatToDila_image_sieve_le_singleton`	
[5, Definition 3.6] ($Cat_{\mathcal{C}}^{\Sigma\text{-reg}}$)	`IsSigmaRegular`	a `Prop` (membership only), not a category; see §5.2.
[5, Fact 3.7] (Θ is Σ-regular)	`CatToDila_isSigmaRegular`	

Mathematics	Lean declaration	Comments
[5, Fact 3.8] ($\Sigma' \subset \Sigma$, L faithful $\Rightarrow$ L' faithful)	`faithful_of_comp_faithful`, `faithful_of_comp_faithful_gen`	replaces the paper's triangle-of-functors argument with the purely categorical fact "if $p \ggg e$ is faithful then p is faithful," which also drives [5, Fact 3.7]; applied instance: `IsSigmaRegular_sum_inl_of` ([5, Proposition 3.15]).
[5, Remark 3.9] (Buan–Marsh)	—	not formalized, and not needed: the paper itself states it is not used in the rest of the text.
[5, Theorem 3.10] (universal property)	`DilaLift` (F'), `DilaLift_fac`, `DilaLift_unique`, `Dila_universal_property`	`DilaLift` is F' as a named construction (a quotient-lift of the path-lift H, §5); `DilaLift_fac` is $F' \circ \Theta = F$, `DilaLift_unique` its uniqueness, and `Dila_universal_property` their packaging as `∃! (G : Dila Z ⇒ D), CatToDila Z ⋙ G = F`, under hypotheses `hfaith : (ImageCenterLocalizationFunctor Z F).Faithful` and `hsieve : ∀ i, Sieve.functorPushforward F (Z.N i) ≤ Sieve.generate (Presieve.singleton (F.map (Z.mor i)))`, i.e. conditions (2),(1) kept separate rather than bundled as membership in $Cat_{\mathcal{C}}^{\Sigma\text{-reg}}$; see §5.2.

Mathematics	Lean declaration	Comments
[5, Fact 3.11] (pushforward of a generated sieve)	`CatToDila_comp_image_sieve_le_singleton`	the composite-functor form used directly in the proof of [5, Theorem 3.10] and [5, Proposition 3.12]; the bare statement about an arbitrary functor H is Mathlib's `Sieve.functorPushforward_comp`.
[5, Proposition 3.12] (Θ represents $Cat_{\mathcal{C}}^{\Sigma\text{-reg}} \to Set$)	`CatToDila_represents`	stated as an "iff": `(∃! G, CatToDila Z ⋙ G = F)` $\leftrightarrow$ `∀ i, ...`, rather than as a bijection of `Hom`-sets; equivalent, more convenient to use.
[5, Fact 3.13] (subsieve $M_i \subset N_i$ gives a faithful φ)	`Fact313Phi`, `IsSigmaRegular_altSieve`	built from [5, Theorem 3.10] applied to Θ, exactly as `restrictPhi`; faithfulness via `Dila_factor_unique` and `faithful_of_comp_faithful` (§7).
[5, Proposition 3.14] ($K \subset I$, functor Φ; full/faithful)	`Center.restrict`, `restrictPhi`, `restrictPhi_full`, `restrictPhi_faithful`	part (i) is `restrictPhi_full` (hypothesis `hI : ∀ i ∉ K, Z.N i = Sieve.generate (Presieve.singleton (Z.mor i))`); part (ii) is `restrictPhi_faithful` (hypothesis `(baseRestrictFunctor Z K hK).Faithful`).
[5, Proposition 3.15] (combining two centers)	`Center.sum`, `CenterZW`, `Phi315` (Φ), `Beta315` (β), `Alpha315` (α), `Alpha'315` (α'), `Iso315`	(i) `Phi315_isSigmaRegular`; (ii) a hypothesis, not a proved lemma (§8.2); (iii) `Alpha315`/`Alpha315_spec`/`Alpha315_unique`; (iv) `Alpha'315`/`Alpha'315_spec`; (v) `Phi315_comp_Alpha315`/`Alpha315_comp_Alpha'315`/`Alpha'315_comp_Alpha315`; (vi) `Iso315`.

Mathematics	Lean declaration	Comments
Remark (direct computation for 3.15(vi))	—	not formalized; an alternative proof strategy the paper explicitly does not use either.
[5, Fact 3.17] (union of pushforward sieves)	`Sieve.functorPushforward_union`	Mathlib.
[5, Proposition 3.18] ($N_i'' = N_i \cup N_i'$)	`Center.sieveUnion`, `Center.altSieve`, `Alpha318`, `Alpha'318`, `Iso318`	mirrors `Alpha315`/`Alpha'315`/`Iso315`, without the extra `hreg` hypothesis (regularity holds unconditionally here).

A.4. **Section 4: codilatations.** Codilatations are formalized exactly as the paper constructs them: by passing to $\mathcal{C}^{\mathrm{op}}$, forming a dilatation there, and passing back; every declaration below is a thin wrapper around the corresponding dilatation declaration of §A.2–A.3, composed with Mathlib's opposite-category API (`Functor.op`, `Functor.rightOp`, `Opposite.op`).

Mathematics	Lean declaration	Comments
[5, Definition 4.1] (cocenter $\{[V_i, d_i]\}_{i\in I}$)	`Cosieve`, `Cocenter`	`Cosieve X` mirrors `Sieve` but is stable under postcomposition; `Cocenter` records `V i : Sieve (Opposite.op (dom i))` directly, already through the identification of [5, Fact 4.2], avoiding re-deriving it at every use.
[5, Fact 4.2] (cosieve from X = sieve over $\mathrm{op}\, X$ in $\mathcal{C}^{\mathrm{op}}$)	`Cosieve.toSieveOp`, `Sieve.toCosieveUnop`, `Cosieve.equivSieveOp`, `Cocenter.toCenterOp`	an explicit `Equiv` `Cosieve.equivSieveOp`, not two one-directional maps; `Cocenter.toCenterOp co : Center C^op` bundles the cocenter under this identification, ready to feed into `Dila`.
[5, Definition 4.3] (codilatation $\mathcal{C}[\{V_i \circ (d_i)^{-1}\}]$)	`Codila`	`Codila co := (Dila (co.toCenterOp))^op`, verbatim the paper's formula.

Mathematics	Lean declaration	Comments
[5, Fact 4.4] ($CoS_E^{\mathcal{C}} = S_E^{\mathcal{C}^{op}}$)	`Cosieve.generate_toSieveOp`	
[5, Proposition 4.5(i)] ($\Upsilon : \mathcal{C} \to \mathcal{C}[\{V_i \circ (d_i)^{-1}\}]$)	`Cocenter.Upsilon`	`:= (CatToDila (co.toCenterOp)).rightOp`.
[5, Proposition 4.5(ii)] (faithful $\mathcal{C}[\{V_i \circ (d_i)^{-1}\}] \to \mathcal{C}[\{d_i\}^{-1}]$)	`Codila.faithful_to_loc_op`	an instance, obtained directly from `Fact_2_14` applied in $\mathcal{C}^{op}$.
[5, Proposition 4.5(iii)] ($\Upsilon \in Cat_{\mathcal{C}}^{\{d_i\}\text{-reg}}$)	`Cocenter.Upsilon_isSigmaRegular`	
[5, Proposition 4.5(iv)] (Υ represents)	`Cocenter.represents`	obtained from `CatToDila_represents` ([5, Proposition 3.12]) applied in $\mathcal{C}^{op}$, combined with [5, Fact 4.4].

A.5. **Section 5: examples.** Rows follow the paper's order: §5.1 (the universal property of localizations recovered from [5, Theorem 3.10]), §5.2 (dilatations of commutative rings, [5, Proposition 5.1], with the ring-level dilatation vendored from [4]'s own Lean development), §5.3–5.4, and §5.5 (Theorem 11.1).

Mathematics	Lean declaration	Comments
§5.1 (universal property of localizations recovered)	`Center.ofMorphisms`, `Center.ofMorphisms_universal_property`, `Fact_2_15`	the center with all sieves trivial (`N := fun _ => ⊤`); any F inverting every d_i factors uniquely, i.e. [5, Proposition 2.8] as a special case of [5, Theorem 3.10].
$\{[M_i, a_i]\}_{i \in I}$ (ring center, [4])	`Multicenter A'`	vendored from [4]'s own formalization.
$A[\{M_i/a_i\}_{i \in I}]$	`A'[M]` (`Multicenter.Dilatation`)	
$\mathcal{C}$ attached to A (single object)	`SingleObj A'`	Mathlib.

Mathematics	Lean declaration	Comments
center in $\mathcal{C}$ attached to $\{[M_i, a_i]\}$	`centerOfMulticenter`	builds a `Center (SingleObj A')` from a `Multicenter A'`.
functor $\mathcal{C} \to \mathcal{C}[\{M_i/a_i\}]$	`toDilatationFunctor`	
[5, Proposition 5.1]	`prop_5_1`, `Iso51`	`prop_5_1` is the universal-property half, via [5, Theorem 3.10] (faithfulness from the generators of M being non-zero-divisors in $A'[M]$); `Iso51` assembles it with its inverse `Psi51` into `Dila (centerOfMulticenter M)` $\cong$ `SingleObj A'[M]`.
naive $\mathcal{C}[\{(a_i)^{-1} \circ M_i\}_{i\in I}] \cong A[M]$ (false, §10.2)	`centerNaive`, `PhiNaive`, `target_elt_not_in_range`, `no_C_compatible_equiv`	`centerNaive` is the literal i-indexed center; `PhiNaive` the comparison functor from [5, Theorem 3.10]; `target_elt_not_in_range` shows $(X+2)/2$ is unreachable; `no_C_compatible_equiv` concludes no compatible functor is an equivalence.
§5.3 (monoid dilatations), §5.4 (pre-additive categories)	—	no numbered statements; not formalized as standalone constructions.
$\mathcal{C}$ $(X, Y, Hom(X,Y) = \{a, b\})$ (§5.5)	`Obj`, `CHom`, `CHom.comp`	two-constructor `inductive Obj | X | Y`, and `CHom : Obj → Obj → Type` with constructors `idX, idY, a, b`; composition by pattern-matching, associativity by `cases ... <;> rfl`.
$\Gamma = \{b\}$	`Gamma`	`MorphismProperty Obj`.

Mathematics	Lean declaration	Comments
$\mathcal{C}[\Gamma^{-1}]$, distinctness of $b, a, ab^{-1}a$	`FSep`, `FSep'`, `cMor`, `pairwise_distinct`	separated by a functor to `SingleObj (Multiplicative ℤ)` sending $a \mapsto 1$, $b \mapsto 0$: witnesses the infinitely many morphisms $X \to Y$ and gives a separating invariant for the three named ones.
$\mathcal{D}$ ($Hom(X,Y) = \{b, a, ab^{-1}a\}$)	`DObj`, `DHom`, `DHom.comp`	same pattern as `Obj`/`CHom`, with a third generator `c` standing for $ab^{-1}a$.
functors $\mathcal{C} \to \mathcal{D}$, $\mathcal{D} \to \mathcal{C}[\Gamma^{-1}]$	`CtoD`, `DtoLoc`, `CtoD_comp_DtoLoc`	identity on objects, as in the paper; `CtoD_comp_DtoLoc` is the commuting triangle.
faithfulness of $\mathcal{D} \to \mathcal{C}[\Gamma^{-1}]$	`DtoLoc_faithful`	
"$\mathcal{C}' \neq \mathcal{D}$ for every center"	`GoodCenter`, `false_of_not_good`, `exists_C_mor_of_good`, `CatToDila_full_of_good`, `no_realizing_center`	see §11 for the two-case analysis.

Appendix B. Erratum to [5]

- The identification $\mathcal{C}[\{(a_i)^{-1} \circ M_i\}_{i \in I}] \cong A[M]$ of [5, Proposition 5.1] should be replaced by the triple identification $\mathcal{C}[\{(a^\nu)^{-1} \circ L^\nu\}_{\nu \in \mathbb{N}^{(I)}}] \cong A[\{L^\nu/a^\nu\}_{\nu \in \mathbb{N}^{(I)}}] \cong A[\{M_i/a_i\}_{i \in I}] = A[M]$ of Theorem 10.1; see §10.1 above.
- [5, Proposition 3.15]'s part (2) is not rigorously proved, and parts (3), (5), (6), which are built on it, inherit the same gap; we essentially turn (2) into an assumption in the formalization (`hreg`, Remark 8.1); see §8.2 above.

Up to these adjustments, [5] is correct and formalized.

Statements and declarations.

Competing interests. The author has no competing interests to declare.

Data availability. The Lean 4 source code underlying this work is openly available at `https://github.com/rndmx/DilCat`.

This work was carried out during the 2025–2026 academic year.

AI use disclosure. Claude (Anthropic) assisted with this matter, under the author's direction.

References

[1] A. Dubouloz, A. Mayeux and J.P. dos Santos, A survey on algebraic dilatations, *Fields Institute Monographs*, Springer, to appear.

[2] P. Gabriel and M. Zisman, *Calculus of fractions and homotopy theory*, Ergebnisse der Mathematik und ihrer Grenzgebiete 35, Springer-Verlag, New York, 1967.

[3] The mathlib Community, The Lean mathematical library, in *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, ACM, 2020, 367–381.

[4] A. Mayeux, Multi-centered dilatations, congruent isomorphisms and Rost double deformation space, *Transformation Groups*, 2024.

[5] A. Mayeux, Dilatations of categories, *Higher Structures*, 9(2), 2025, 62–75.

[6] A. Mayeux, Formalizing all indexed mathematics as a benchmark for general reasoning, in *Intelligent Systems and Applications — Proceedings of the 2026 Intelligent Systems Conference (IntelliSys)*, Lecture Notes in Networks and Systems, Springer, to appear, arXiv:2606.03835.

[7] A. Mayeux, T. Richarz and M. Romagny, Néron blowups and low-degree cohomological applications, *Algebraic Geometry*, 10(6), 2023, 729–753.

[8] A. Mayeux and J. Zhang, Formalizing multi-graded Brenner–Schröer Proj schemes and dilatations of rings in Lean4, arXiv:2606.01438, 2026.

[9] L. de Moura and S. Ullrich, The Lean 4 theorem prover and programming language, in *Automated Deduction — CADE 28*, Lecture Notes in Computer Science 12699, Springer, 2021, 625–635.

University of Wisconsin–Madison, Madison, WI, USA
Email address: mayeux@wisc.edu